

RENANCE

FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN

CSC 224 — Past Questions & Materials
by Renance

Course: CSC 224
Institution: Federal University of Petroleum Resources, Effurun
Papers in this pack: 5
Levels: 200

This study pack was compiled and restructured by Renance for CSC 224 students at Federal University of Petroleum Resources, Effurun. It brings together 5 past paper(s)/material(s) into one clean, readable document so you can revise without chasing scattered PDFs. Practise these same questions interactively on Renance with instant answers, AI explanations, and full CBT mock exams — over 235,000+ questions across JAMB, Post-UTME, WAEC, NECO and university courses.

Study online: renance.app
Get the app: Google Play — [dev.renance.renance](https://play.google.com/store/apps/details?id=dev.renance.renance)

Generated by Renance AI

Paper 1 of 5 — CSC 224 Operating system 2023/2024

Level 200 • Second Semester • PastQuestion

This source paper is a scanned document; its pages are reproduced below.



COLLEGE OF PETROLEUM RESOURCES, EFFURUN
FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN
2023/2024 SECOND SEMESTER EXAMINATION

COURSE CODE/TITLE: CSC224 – OPERATING SYSTEM I

INSTRUCTIONS

- (i) Answer **FOUR** questions.
- (ii) Time allowed: **2Hrs, 10mins**
- (iii) Do not mix up questions. Start a new question on a fresh page.
- (iv) Use the same ink colour throughout. Otherwise, the invigilator should sign
- (v) Submit your question paper along with your answer booklet

Question One

- (a) (i) Describe what could happen if the operating system kernel is removed from the operating system and installed on a computer system (5mks)
- (ii) Explain briefly the danger(s) of an operating system running solely on user mode and state how modern operating systems tend to avert such danger(s). (5mks)
- (b) (i) List any two conditions that could cause an interrupt in operating system (4mks)
- (ii) Explain concisely, the role of the *interrupt vector* in the handling of interrupt in operating system (4mks)

Question Two

- (a) Briefly explain the two key purposes of the operating system (6mks)
- (b) Explain the roles of the following in the execution of the operating system
 - (i) BIOS (3mks)
 - (ii) Master Boot Record (MBR) (3mks)
- (c) Briefly explain any three operating system features needed for multiprogramming (6mks)

Question Three

- (a) The Windows operating system provides both GUI and Command Line Interfaces (CLI). Hence, describe how to access the CLI on a windows system (4mks)
- (b) State and illustrate the command to create a directory called *csc224Exam* on a command line interface of MS DOS. The *csc224Exam* also contains two sub directories called *test* and *assignment* respectively (8mks)
- (c) List three functions of the operating system with respect to managing secondary storage devices (6mks)

Question Four

- (a) Explain the functions of the clock interrupt in operating system (6mks)
- (b) Describe succinctly, the layered structured operating system and give two reasons why you think this approach is an improvement over the monolithic structured operating systems (8mks)
- (c) Explain, in few words, why user applications are not able to execute privileged instructions except the operating system (4mks)

Question Five

- (a) Assume you were asked to design a general-purpose operating system. Give two principles you could adopt in your design and explain how you intend to apply each of the principles during your design (8mks)
- (b) Differentiate between *mechanism* and *policy* with respect to operating system design (4mks)
- (c) One of the design goals of the operating system is to ensure isolation of critical resources. Describe how you can achieve this goal in the design of a multi-tasking/multi-user operating system (6mks)

Question Six

- (a) Briefly explain why microkernel structured Operating System such as Windows NT performs slowly despite its small size kernel? (7mks)
- (b) Explain concisely how hardware technology affected the design of MS-DOS (4mks)
- (c) In practice, very few Operating Systems adopt a strictly single defined structure. Explain concisely, why this statement is true using Linux or Windows Operating System as example (7mks)

Page 1 of 1

Paper 2 of 5 — CSC 224 PQ 2024

Level 200 • PostUTME • PastQuestion

DEPARTMENT OF COMPUTER SCIENCE

=! COLLEGE, OF SCIENCE

FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EPFURUN

2023/2024 SECOND SEMESTER EXAMINATION

Bee here FOUR acto

} Time allowed: 2Hrs, 10mins

1) Do not mix up questions. Start a new question on.» fest) pay,

) Use the same ink colour throughout, Otherwise, the invijyiliti should sig

} Submit your question paper along with your umwer booklet

bestion One

(a) (7) Describe whut could happen if the operating system kemel x removed from the operating eystem and

installed on a computer system (Sma)

(ii) Explain briefly the danger(s) of an operating system running solely on user made and state how meders

Operating systems tend to avert such danger(s), (Sms)

(b) (i) List any two conditions that could cauye an interrupt jo operating aystert (4emkay

(ii) Explain concisely, the role of the interrupt vitor in the handling of interrught in opetating ster Aukay |

westion Two

{a) Briefly explain the two key purposes of the operating system (heauky)

(b) Explain the roles of the following in the executlon of the operating system

17) BIOS (dunks)

(ii) Master Boot Recont (MBR) (lorky

(¢) Briefly explain any three operating system features needed for multiprogeammliyg (Gms)

uestion Three

(a) The Windows operting system provides beth GUI and Conunand Line triterfiacies (C11). Hence, descrihe how

to access the CL! on a windows system (4imks)

(b) State and illustrate the command to create a directary culled cre24Eawr on = command fine imeriace of MS

DOS. The cic224Evam also contains two sub directories called heat and analyronent respectively (Menke)

(c) Lint three functions of the operating system with respect to managing seconciery storage devices (Genks)

uestion Four

(a) Explain the functions of the clock interrupt In operating sysivin (Geinks)

(b) Describe succinctly, the layered structured operating system and give two reasons why you think this

approach is an improvement over the monolithic: structured «operating system» (Marks)

(c) Explain, in few words, why user applications are not able to execute privileged instructions except the

operating system (4 marks)

Question Five

Assume a general-purpose operating system, Give two principles you could in

the poem Irenes po Meigs jet tramps as if in

between mechanism and policy with respect to operating system design (Aris)

operation is to ensure solution of exit

the program/multitasking

Paper 3 of 5 — CSC 224 PQ 2019

Level 200 • Second Semester • PastQuestion

This source paper is a scanned document; its pages are reproduced below.



DEPARTMENT OF MATHEMATICS AND COMPUTER SCIENCE
COLLEGE OF SCIENCES
FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN
2018/2019 SECOND SEMESTER EXAMINATION

CSC224 – OPERATING SYSTEM I

INSTRUCTIONS

- (i) Answer FOUR questions, *two from each section*. All questions carry equal marks
(ii) Time Allowed: 2½Hrs.
(iii) Do not mix up questions. Start a new question on a fresh page.

Section A: Answer any two questions only from this section

Question One

- (a) (i) What is a System Bus? (2mks) (ii) List and explain the three main types of I/O busses (4mks)
(b) With the aid of a well labelled diagram, explain the relationship between CPU, Program Counter, Registers and Main Memory (4mks)
(c) Mention and explain five (5) functions of an Operating System. (7½mks)

Question Two

- (a) A process executing on an Operating system tries to access a page of memory that is not allocated at the moment. This results in an error condition. Classify or categorise this error condition and discuss the steps the Operating System will take to resolve this condition. (4½mks)
(b) (i) What are System calls (2mks) (ii) List four categories of System calls and give examples (6mks)
(c) Write short notes on the following (i) Monolithic Structure (ii) Microkernel Structure (iii) Layered Structure (5mks)

Question Three

- (a) Using diagrams where necessary, discuss the linear sequence followed by Von Neumann architecture in executing instructions. (4mks)
(b) (i) What are Interrupts (2mks) (ii) Discuss the various types of Interrupts (4mks)
(c) An Operating System provides services to both the users and to the programs. Mention and explain five (5) of the services. (7½mks)

Section B: Start this section at the middle of your booklet
Answer any two questions only from this section.

Question Four

- (a) A Process is an instance of a program in execution. Give three attributes that help to describe such a Process (3mks)
(aii) Concurrency involves running multiple Processes on a single Processor. Describe two Operating System design concepts that make this possible (7mks)
(b) With the aid of a diagram, describe the Process design concept that enables Processes to be protected from one another (7½mks)

Question Five

- (a) With the aid of diagram, describe the Process Control Block (PCB) of a Process (6mks)
(aii) Give two reasons why you think the access to PCB should be protected (4mks)
(b) With the aid of a queuing diagram, describe the five state Process model (7½mks)

Question six

- (a) Discuss succinctly the following Process scheduling algorithms (7½mks)
(i) First Come First Serve (FCFS) (ii) Shortest Job First (SJF)
(aii) Outline two criteria for achieving optimum Process scheduling (4mks)
(b) Four Processes are to be scheduled for processing using the pre-emptive SJF algorithm. Their arrival time and burst time are shown in the table below. Using a Gantt chart, compute the total and average wait time of the processes (6mks).

Process	P ₁	P ₂	P ₃	P ₄
Arrival time	0	1	2	3
Burst time	8	4	9	5

(iii) Hence, determine the vector whose image is (2, 1, 5).

Paper 4 of 5 — CSC224 LNotes 2021 1784049866428 0

Level 200 • Second Semester • Handout

CSC224: Operating System I

Department of Computer Science

Federal University of Petroleum Resources, Effurun,

Delta State

OPERATING SYSTEM I

[CSC224]

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 1

CSC224: Operating System I

COURSE GOAL: To provide the students with the fundamental knowledge of operating system necessary to prepare them for future courses in operating system and to apply them in real world situations.

BEHAVIOURAL OBJECTIVES:

At the end of the course, students should be able to:

- i. Define Operating System and discuss its functions
- ii. Discuss the hardware support for operating system
- iii. Describe how the operating system operates
- iv. Categorize Operating System
- v. List and explain the major components of Operating System
- vi. Describe the services an Operating System provides to users, processes and other systems
- vii. Discuss the various ways of structuring an Operating System
- viii. Enumerate and discuss the design consideration of operating system
- ix. Discuss the design goals and architecture of some operating systems
- x. Explain why Operating System are sometimes difficult to design
- xi. Describe the various features of processes, including scheduling, creation, and termination
- xii. Describe communication among processes
- xiii. Explain the concept of race condition and critical sections in Processes/ threads
- xiv. Examine some strategies that provide implicit threading
- xv. Explain the benefits and challenging issues of Threads
- xvi. Differentiate between processes and threads

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 2

CSC224: Operating System I

COURSE OUTLINE

1. Meaning, functions, and support for Operating System

- i. What is Operating system?
- ii. Operating system services
- iii. Hardware support for operating systems
- iv. Execution of the Operating system
- v. Classification of operating system
- 2. Organization & Design of Operating System
 - i. Operating system functional components
 - ii. Operating system design consideration
 - iii. Operating system design Approaches
 - a. Monolithic e. Modules
 - b. Layered f. Hybrid
 - c. Microkernel g. Virtual machines
 - d. Client 3 Server model
 - iv.. Mobile Operating Systems
- 3. Processes
 - i. What are Processes? vi. Process Creation and
 - ii. Process Attributes Termination
 - iii. Process Control Block vii. Fundamentals of Inter-
 - iv. Process State Models Process Communication
 - v. Fundamentals of Process viii. Communication in client
 - Scheduling 3 server systems
- 4. Threads
 - i. Types of Threads iv. Issues with Threads
 - ii. Multi-Threading Models v. Processes Vs Threads
 - iii. Benefits of Threads

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 3

CSC224: Operating System I

LECTURE SCHEDULE

Week Topic Sub-topics

- 1 Course Orientation Syllabus and Policies
- 2-3 Meaning, functions, and i. What is Operating system?
support for Operating System ii. Operating system services
- iii. Hardware support for operating systems
- iv. Execution of the Operating system
- v. Classification of operating system
- 4-6 Organization & Design of i. Operating system functional components
Operating System ii. Operating system design consideration
- iii. Operating system design Approaches
- iv. Mobile Operating System

- 7-9 Processes i. What are Processes?
- ii. Process Attributes
- iii. Process Control Block
- iv. Process State Models
- v. Fundamentals of Process Scheduling
- vi. Process Creation and Termination
- vii. Fundamentals of Inter-Process Communication
- viii. Communication in client 3 server systems
- 10 - 11 Threads i. Types of Threads
- ii. Multi-Threading Models
- iii. Benefits of Threads
- iv. Issues with Threads
- v. Processes Vs Threads
- 12 Revision

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 4

CSC224: Operating System I

TEXTBOOK

Operating System Concepts, Abraham Silberchatz, Peter B. Galvin, Greg Gagne, Ninth Edition, 2012, Wiley.

Modern Operating Systems, Andrew S. Tanenbaum, Herbert Bos Pearson Education, Inc Fourth Edition 2008

CONTACTS:

GRADING

1. CA (30mks):

- (i) Assignments-----12mks (or 0 for copying)
- (ii) Attendance.....3mks (70% total attendance)
- (iii) Participation.....3mks
- (iv) Test-----12mks (or 0 for cheating)

2. Exam (70mks)

POLICIES

(i) Attendance - It is expected that every student will be in class for lectures. Attendance records will be kept and used to award marks for each student. Only students with up to or above 75% attendance will be awarded 3mks. Signing for friends is an act of DISHONESTY and it is HIGHLY PROHIBITED.

(ii) Makeup test - There shall be no makeup test. The date for the test shall be announced to the class at least one week before the test.

(iii) Academic Integrity – Any form of academic dishonesty, such as copying during test /exam or making copies of other students' work, is prohibited.

(iv). Submission of Assignments/Lab work – Late submission of assignment or lab work will be graded as Score – x, where x is the number of days of late submission..

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 5

CSC224: Operating System I

1. Meaning, functions, and support for Operating System

1.1 What is Operating System?

An Operating System (OS) is a software that manages the computer hardware, as well as providing an environment for application programs to run.

□ That is the Operating System acts as an interface between a user of the computer and the computer hardware and also provides an environment in which we can execute programs.

The purpose of an operating system involves two key goals:

- Availability of a convenient, easy-to-use, and powerful set of services that are provided to the users and the application programs in the computer system
- Management of the computer resources in the most efficient manner

Operating systems are virtually present in almost everything we use today including toasters, cars, ships, spacecraft.. They are the basis for game machines, music players, cable TV tuners, and industrial control systems. A computer system has many hardware and software resources that may be required to solve a problem: CPU time, memory space, files storage space, I/O devices etc.

□ The operating system acts as the manager of these resources

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 6

CSC224: Operating System I

1.2 Operating System Services

An Operating System provides services to both the users and to the programs.

- It provides programs an environment to execute.
- It provides users the services to execute the programs in a convenient manner.

Following are a few common services provided by an operating system:

- i. User interface vi. Error Detection
- ii. Program execution vii. Resource Allocation
- iii. I/O operations viii. Protection
- iv. File System manipulation ix. Accounting
- v. Communication

(i) User interface: Almost all operating systems have a user interface (UI).

This interface can take several forms.

- One is a command-line interface (CLI), which uses text commands and a method for entering them (say, a keyboard for typing in commands in a specific format with specific options).
- Another is a batch interface, in which commands and directives to control those commands are entered into files, and those files are executed.
- Most commonly, a graphical user interface (GUI) is used. Here, the interface is a window system with a pointing device to direct I/O, choose from menus, and make selections and a keyboard to enter text.
- Some systems provide two or all three of these variations.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 7

CSC224: Operating System I

(ii) Program Execution

Operating systems handle many kinds of activities from user programs to system programs like printer spooler, name servers, file server, etc.

- Each of these activities is encapsulated as a process.
- A program that is loaded into memory and is executing is commonly referred to as a process.□

The following are the major activities of an operating system with respect to program management:

- Loads a program into memory
- Handles program's execution
- Provides a mechanism for process synchronization
- Provides a mechanism for process communication
- Provides a mechanism for deadlock handling

(iii) I/O Operation

An I/O subsystem comprises of I/O devices and their corresponding driver software.

Drivers hide the peculiarities of specific hardware devices from the users.

An Operating System manages the communication between user and device drivers.

- I/O operation means read or write operation with any file or any specific I/O device.
- Operating system provides the access to the required I/O device when required.

(iv). File System Manipulation

A file represents a collection of related information. Computers can store files on the disk (secondary storage), for long-term storage purpose.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 8

CSC224: Operating System I

- o Examples of storage media include magnetic tape, magnetic disk and optical

disk drives like CD, DVD.

- o Each of these media has its own properties like speed, capacity, data transfer rate and data access methods.

A file system is normally organized into directories/folders for easy navigation and usage. These directories/folders may contain files and other directories/folders.

The following are the major activities of an operating system with respect to file management:

- Program needs to read a file or write a file.

- o The operating system gives the permission to the program for operation on file.

- o Permission varies from read-only, read-write, denied, and so on.

- Operating System provides an interface to the user to create/delete files.

- Operating System provides an interface to the user to create/delete directories/folders.

- Operating System provides an interface to create the backup of file system.

(v) Communication

In case of distributed systems which are a collection of processors that do not share memory, peripheral devices, or a clock, the operating system manages communications between all the processes. Multiple processes communicate with one another through communication lines in the network.

The following are the major activities of an operating system with respect to communication:

- Two processes often require data to be transferred between them.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 9

CSC224: Operating System I

- o Both the processes can be on one computer or on different computers, but are connected through a computer network.

- Communication may be implemented by two methods, either by Shared Memory or by Message Passing.

- The operating system manages communications between all the processes

(vi). Error Handling

Errors can occur anytime and anywhere. An error may occur in CPU, in I/O devices or in the memory hardware.

The following are the major activities of an operating system with respect to error handling:

- The OS constantly checks for possible errors.

- The OS takes an appropriate action to ensure correct and consistent computing.

(vii). Resource Management

In case of multi-user or multi-tasking environment, resources such as main memory, CPU cycles and files storage are to be allocated to each user or job.

The following are the major activities of an operating system with respect to resource management:

Manage secondary storage devices

Allocate appropriate amount of disk space when files are created

Deallocate space when files are removed

Ensure that a new file does not overwrite an existing file Schedule disk requests

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 10

CSC224: Operating System I

Manage primary storage

Allocate appropriate amount of memory space when programs are to be loaded into the memory for execution

Deallocate space when processes terminate

Ensure that a new process is not loaded on top of an existing process

Ensure that a process does not access memory space that does not belong to it Minimize the amount of unused memory space

Allow execution of programs larger in size than the available main memory

Manage processes

Allow simultaneous execution of processes by scheduling the CPU(s)

Prevent deadlocks between processes

Ensure integrity of shared data

Synchronize executions of cooperating processes

(viii). Protection

Considering a computer system having multiple users and concurrent execution of multiple processes, the various processes must be protected from each other's activities.

□ Protection refers to a mechanism or a way to control the access of programs, processes, or users to the resources defined by a computer system.

The following are the major activities of an operating system with respect to protection:

□ The OS ensures that all access to system resources is controlled.

□ The OS ensures that external I/O devices are protected from invalid access attempts.

□ The OS provides authentication features for each user by means of passwords.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 11

CSC224: Operating System I

(ix) Accounting. An operating system collects usage statistics for various resources and monitor performance parameters such as response time.

o On any system, this information is useful in anticipating the need for future enhancements and in tuning the system to improve performance.

o On a multiuser system, the information can be used for billing purposes.

1.3. Hardware Support for Operating Systems

Modern computers are built on the Von Neumann architecture. The Von Neumann architecture uses a single processor which follows a linear sequence of fetch-decode-execute.

□ In order to do this, the processor has to use some special registers, which are discrete memory locations with special purposes attached.

The following table shows these registers and their functions:

Register Function

Program Counter (PC)

Keeps track of where to find the next instruction so that a copy of the instruction can be placed in the current instruction register. Sometimes the program counter is called the Sequence Control Register (SCR) as it controls the sequence in which instructions are executed.

Current Instruction Register (CIR) Holds the instruction that is to be executed

Memory Address Register (MAR) Used to hold the memory address that contains either the next piece of data or an instruction that is to be used.

Memory Data Register (MDR) Acts like a buffer and holds anything that is copied from the memory ready for the processor to use it.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 12

CSC224: Operating System I

Index Register (IR) Used for modifying operand addresses during the run of a program, typically for doing vector/array operations.

Accumulator (ACC) Holds results

The following diagram shows the layout of the registers in the CPU and their data paths

Memory

CPU

Control Unit ALU

PC Accumulator

CIR

MAR

MDR

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 13

CSC224: Operating System I

1.4. Executing the Operating System

1.4.1 Operating System Modes

A typical operating system has at least two (dual) modes of operation: user mode and kernel mode (also called supervisor mode, system mode, or privileged mode).

□ A bit, called the mode bit (a bit in a special register called the processor status word - PSW), is added to the hardware of the computer to indicate the current mode: kernel (0) or user (1).

□ User applications run in user mode.

The heart of the operating system is called the kernel and provides a collection of functions that perform the basic services such as scheduling applications.

o The kernel runs in kernel mode.

o At system boot time, the hardware starts in kernel mode.

o The operating system is then loaded and starts user applications in user mode.

Various CPU instructions are only available to software running in kernel mode, i.e., when the bit is set. Hence these privileged instructions can only be executed by the operating system, and not by user applications.

Examples privileged instructions include:

- Instructions to set the interrupt priority level (IPL). This can be used to block certain classes of interrupts from occurring, thus guaranteeing undisturbed execution.
- Instructions to set the hardware clock to generate an interrupt at a certain time in the future.
- Instructions to activate I/O devices. These are used to implement I/O operations on files.
- Instructions to load and store special CPU registers, such as those used to define the accessible memory addresses, and the mapping from each application's virtual addresses to the appropriate addresses in the physical memory.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 14

CSC224: Operating System I

- Instructions to load and store values from memory directly, without going

through the usual mapping. This allows the operating system to access all the memory.

The dual mode of operation provides us with the means for protecting the operating system from errant users and errant users from one another.

- o This protection is accomplished by designating some of the machine instructions that may cause harm as privileged instructions, such as those listed above.

The hardware allows privileged instructions to be executed only in kernel mode.

- If an attempt is made to execute a privileged instruction in user mode, the hardware does not execute the instruction but rather treats it as illegal and traps it to the operating system.

The lack of a hardware-supported dual mode can cause serious shortcomings in an operating system.

- o For instance, MS-DOS was written for the Intel 8088 architecture, which has no mode bit and therefore no dual mode.

- o A user program running awry (improperly) can wipe out the operating system by writing over it with data; and multiple programs are able to write to a device at the same time, with potentially disastrous results.

Modern versions of the Intel CPU do provide dual-mode operation. Accordingly, most contemporary operating systems such as Microsoft Windows 7, as well as Unix and Linux take advantage of this dual-mode feature and provide greater protection for the operating system.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 15

CSC224: Operating System I

- How to Enter the Kernel Mode:

There are two ways to enter kernel mode: interrupts and system calls.

(a) Interrupts: Interrupts are special conditions that cause the CPU not to execute the next instruction.

- Instead, it enters kernel mode and executes an operating system interrupt handler.

- Modern operating systems are interrupt driven.

But how does the CPU (hardware) know the address of the appropriate kernel function?

- When the system is booted, the operating system stores the addresses of the interrupt handling functions in the interrupt vector.

- When an interrupt occurs, the hardware stores the current PSW (Processor Status Word) and PC and loads the appropriate PSW and PC values for the interrupt handler.

- o The PC (program counter) is a special register that holds the address

of the next instruction to be executed.

- o The PSW indicates execution in kernel mode.

- The PC is obtained by using the interrupt number as an index into the interrupt vector, and using the address found there.

- Note that the hardware does this blindly using the predefined address of the interrupt vector as a base.

- The interrupt vector is a predefined location in memory where all the entry points to the operating system are stored.

- It is indexed by interrupt number: the address of the handler for interrupt i is stored in entry i of the vector.

- When a certain interrupt occurs, the hardware uses the interrupt number as an index into the interrupt vector, and loads the value found there into the PC, thus jumping to the appropriate handler.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 16

CSC224: Operating System I

- o It is up to the operating system to actually store the correct addresses in the correct places.

- o If it does not, this is a bug in the operating system.

- Types of Interrupts

There are two main types of interrupts: external (asynchronous) and internal (synchronous).

i. External (Asynchronous) interrupts are generated by external devices at unpredictable times. Examples include:

- o Clock interrupt: This tells the operating system that a certain amount of time has passed.

- Its handler is the operating system function that keeps track of time. S

- Sometimes, this function also calls the scheduler which might pre-empt the current application and run another in its place.

- Without clock interrupts, the application might run forever and monopolize the computer.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 17

CSC224: Operating System I

- o I/O device interrupt: This tells the operating system that an I/O operation has completed. The operating system then wakes up the application that requested the I/O operation.

ii. Internal (synchronous) interrupts occur as a result of an exception condition when executing the current instruction (as this is a result of what the software did, this is sometimes also called a <software interrupt>). This means that the processor

cannot complete the current instruction for some reason, so it transfers responsibility to the operating system. There are two main types of exceptions:

- o An error condition: This tells the operating system that the current application did something illegal (division by zero, arithmetic overflow, trying to issue a privileged instruction, etc.). The handler is the operating system function that deals with misbehaved applications; usually, it kills them.
- o A temporary problem: for example, the process tried to access a page of memory that is not allocated at the moment. This is an error condition that the operating system can handle, and it does so by bringing the required page into memory.

When the handler finishes its execution, the execution of the interrupted application continues where it left off except if the operating system killed the application or decided to schedule another one.

(b) System calls: The second way to enter kernel mode is the system calls. An application can also explicitly transfer control to the operating system by performing a system call.

□ System calls provide the means for a user program to ask the operating system to perform tasks reserved for the operating system on the user program's behalf.

□ This is implemented by issuing the trap instruction.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 18

CSC224: Operating System I

- o This instruction causes the CPU to enter kernel mode, and set the program counter to a special operating system entry point.

The operating system then performs some service on behalf of the application.

Technically, this is actually just another (internal) interrupt but a desirable one that was generated by an explicit request.

Typical system calls include:

- Open, close, read, or write to a file.
- Create a new process (that is, starts running another application).
- Get some information from the system, e.g. the time of day.
- Request to change the status of the application, e.g. to reduce its priority or to allow it to use more memory.

As an operating system can have more than a hundred system calls, the hardware cannot be expected to know about all of them (as opposed to interrupts, which are a hardware thing to begin with).

The sequence of events leading to the execution of a system call is therefore slightly more involved:

- i. The application calls a library function that serves as a wrapper for the system call.
- ii. The library function (still running in user mode) stores the system call identifier and the provided arguments in a designated place in memory.
- iii. It then issues the trap instruction.
- iv. The hardware switches to privileged mode and loads the PC with the address of the operating system function that serves as an entry point for system calls.
- v. The entry point function starts running (in kernel mode). It looks in the designated place to find which system call is requested.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 19

CSC224: Operating System I

- vi. The system call identifier is used in a big switch statement to find and call the appropriate operating system function to actually perform the desired service. This function starts by retrieving its arguments from where they were stored by the wrapper library function.

The diagram below shows the transition from user to kernel mode due to user application request.

1.4.2 Starting an Operating System/ Booting the Computer

On the Parent board (i.e. motherboard).is a program called the system BIOS (Basic Input Output System). – see diagram below

□ The BIOS contains low-level I/O software, including procedures to read the keyboard, write to the screen, and do disk I/O, among other things.

When the computer is powered on, the BIOS is started.

- o It first checks to see how much RAM is installed and whether the keyboard and other basic devices are installed and responding correctly.
- o It starts out by scanning the ISA and PCI buses to detect all the devices attached to them.
- o This process is commonly referred to as Power-On Self Test (POST).

At this stage the computer's CPU is operating in Real Mode (or real address mode),

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 20

CSC224: Operating System I

- o which means that it is only capable of using approximately 1 MB of memory built into the Parentboard

A sample of BIOS in the parentboard

If the hardware is all working, BIOS will then look for a small sector at the very beginning of the primary hard disk called the Master Boot Record (MBR).

- o The MBR contains a list, or map, of all of the partitions on the computer's

hard disk (or disks).

After the MBR is found the Bootstrap Loader (an initial program stored in ROM) follows basic instructions for starting up the rest of the computer, including the operating system.

- o If multiple operating systems are installed, the user will be given a choice of which operating system to use.

The next stage is called Early Kernel Initialization. The Kernel is the core of the operating system, and it regulates all of the background functions of the computer.

- In the Early Kernel Initialization stage, a smaller core of the Kernel is activated.

- This core includes the device drivers needed to use the computer's RAM chips.

- o Without the extra memory provided by RAM, it is not possible to run the more complicated code for the remainder of the operating system.

The procedure of starting a computer by loading the kernel is known as booting the system.

Once the early Kernel Initialization is complete, the CPU switches to Protected Mode.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 21

CSC224: Operating System I

- The computer can now take advantage of the extended memory address system provided by RAM, and the operating system's Kernel is fully initialized.

- Only at this stage are the first User Mode processes initialized, and the user can begin interacting with the operating system, applications and hardware.

The diagram below shows the stages in starting an operating system.

Stages of starting an Operating System

Where are the Operating Systems Stored?

- Some systems such as cellular phones, tablets, and game consoles store the entire operating system in ROM (firmware).

- o Storing the operating system in ROM is suitable for small operating systems, simple supporting hardware, and rugged operation.

- o A problem with this approach is that changing the bootstrap code requires changing the ROM hardware chips.

- o A problem with firmware in general is that executing code there is slower than executing code in RAM.

- Some systems store the operating system in firmware and copy it to RAM for fast execution.

- For large operating systems (including most general-purpose operating systems like Windows, Mac OS X, and UNIX) or for systems that change frequently, the bootstrap loader is stored in firmware, and the operating system is on disk.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 22

CSC224: Operating System I

1.5 Classification of Operating System

Operating System can be classified according to:

- o how user programs are processed
- o the number of users and application they support
- o

1.5.1 Classification according to how user programs are processed

□ Batch Operating Systems: The common input devices were card readers and tape drives. The common output devices were line printers, tape drives, and card punches.

o The users of such systems did not interact directly with the computer systems. Rather, the user prepared a job which consisted of the program, the data, and some control information about the nature of the job (control cards) and submitted it to the computer operator.

o The job would usually be in the form of punch cards. At some later time (perhaps minutes, hours, or days), the output appeared. The output consisted of the result of the program, as well as a dump of memory and registers in case of program error.

To improve utilization, the concept of a batch OS was developed. . The central idea behind the simple batch-processing scheme is the use of a piece of software known as the monitor.

To process user programs,

- the monitor first has to be loaded into memory.
- Then it reads in programs one at a time from the input devices.
- As each program is read in, it will be placed in the user program area of main memory, and control is passed to this program.
- When the execution of the program is completed, it returns control to the monitor, which moves on to process the next program.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 23

CSC224: Operating System I

With this type of OS, the user no longer has direct access to the processor. Instead, the user submits the job on cards or tape to a computer operator, who batches the jobs together sequentially and places the entire batch on an input device, for use by the monitor.

o Each program is constructed to branch back to the monitor when it completes processing, at which point the monitor automatically begins loading the next program.

- The problems with Batch Systems include
- o lack of interaction between the user and job,

o and CPU is often idle because the speed of the mechanical I/O devices is slower than CPU.

□ Multi-programmed Operating Systems: To overcome the problem of underutilization of CPU and main memory, multiprogramming was introduced.

□ The multiprogramming is interleaved execution of multiple jobs by the same computer.

o when one program is waiting for I/O transfer; there is another program ready to utilize the CPU.

□ One problem with multiprogramming is that jobs that come late in the batch job list won't get chance to run until the jobs before them have completed,

o thus their users have to wait a long time to obtain the results.

□ Some programs may even need interaction with users, which requires the processor to switch to these programs frequently.

□ Different forms of multiprogramming operating system are multitasking, multi-process and multiuser operating systems

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 24

CSC224: Operating System I

OS features needed for multiprogramming:

- Memory management 3 the system must allocate the memory to several jobs
- Job Scheduling 3 Choose jobs in the job pool to bring them into memory
- CPU scheduling 3 the system must choose among several jobs ready to run
- Allocation of devices
- Protection (memory and OS protection)

Below is the memory layout for a multiprogramming system

0 Operating System

Job 1

Job 2

Job 3

Job 4

max

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 25

CSC224: Operating System I

□ Time-sharing Operating systems: A time-sharing system is multi-user,

multi-process, and interactive system.

o This means that it allows multiple users to use the computer simultaneously.

o In a time-sharing system, multiple users simultaneously access the system through terminals, with the OS interleaving the execution of each user program in a short burst or quantum of computation.

A user can run one or more processes at the same time and interact with his/her processes.

A time-shared system uses

□ multiprogramming (several programs run concurrently, interleaving their executions over time, which are perceived as simultaneous) and

□ CPU scheduling to provide each user with a small portion of a time-shared computer.

Each user has at least one separate program in memory. To obtain a reasonable response time, jobs may have to be swapped in and out of main memory.

o The Processor's time which is shared among multiple users simultaneously is termed as time-sharing.

o UNIX, Linux, Widows NT server, and Windows 2000 server are timesharing systems.

OS features needed for Time-Sharing include:

- Job Scheduling • Job synchronization and communication
- Memory management • Deadlock avoidance
- Process management • Allocation of devices
- CPU scheduling • Protection

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 26

CSC224: Operating System I

□ Distributed Systems: Distributed system is collection of loosely coupled processors interconnected by a communications network.

□ Distributed systems depend on networking for their functionality.

□ The nodes in a distributed system may vary in size and function. They may include small microprocessors, personal computers, and large general-purpose computer systems.

o These processors are referred to by a number of names, such as processors, sites, machines, and hosts, depending on the context in which they are mentioned.

o We mainly use site to indicate the location of a machine and node to refer to a specific system at a site.

Generally, one node at one site, the server, has a resource that another node at

another site, the client (or user), would like to use.

- There are four major reasons for building distributed systems:
 - o resource sharing, o reliability, and
 - o computation o communication
- speedup,

Distributed Computing Systems commonly use two types of Operating Systems

- o distributed operating system and
- o Network operating. system

Distributed Operating Systems: A distributed operating system is for a multi-user operating system. Users can access other resources and computer on the network in a unified way.

- Every resource is visible on every site. A site is a specific location in a network containing two or more computer systems.
- A distributed operating system provides a less autonomous environment.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 27

CSC224: Operating System I

- o The different computers communicate closely enough to provide the illusion that only a single operating system controls the network.

- Distributed operating system allows programs to run on several processors at the same time.

- o Processors in a distributed system may vary in size and function. These processors are referred as sites, nodes, computers, and so on.

- When a program is executed on a distributed system, user is not aware of where the program is executed or the location of the resources accessed.

Network Operating System Network operating system is for a single-user operating system.

- Users are aware of other resources and computer on the network, and can access them by logging into the appropriate remote host.

- o A host is a specific computer system in the network that provides services.

- o A node is the name assigned to the host to identify it to other computers.

A network operating system is an operating system that provides features such as file sharing across the network, along with a communication scheme that allows different processes on different computers to exchange messages.

- A computer running a network operating system acts autonomously

from all other computers on the network,
 o the users are aware of existence of multiple autonomous computers and can log in to remote machines and copy files from one machine to another machine.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 28

CSC224: Operating System I

A Network Operating System runs on a server and provides the server the capability to manage data, users, groups, security, applications, and other networking functions.

The primary purpose of the network operating system is to allow shared file and printer access among multiple computers in a network, typically a local area network (LAN), a private network or to other networks.

Some of typical characteristics of network operating systems which make it different from distributed operating system are:

- Each computer has its own private operating system instead of running part of a global system wide operating system.
- Each user normally works on his/her own system; using a different system requires some kind of remote login, instead of having the operating system dynamically allocate processes to CPUs.
- Users are typically aware of where each of their files are kept and must move file from one system to another with explicit file transfer commands instead of having file placement managed by the operating system.

Examples of network operating systems include Microsoft Windows Server 2003, Microsoft Windows Server 2008, UNIX, Linux, Mac OS X, Novell NetWare, and BSD (Berkeley System Distribution). Also, Android and iOS, are network operating systems.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 29

CSC224: Operating System I

- Embedded Operating System: Embedded operating systems are designed to be used in embedded computer systems.
- Embedded systems run on the computers that control devices that are not generally thought of as computers and which do not accept user-installed software.
- They are able to operate with a limited number of resources.
- Typical examples are microwave ovens, TV sets, cars, DVD recorders, cell phones, MP3 players.

The main property which distinguishes embedded systems from handhelds is the certainty that no un-trusted software will ever run on it.

o You cannot download new applications to your microwave oven⁴all the software is in ROM.

o This means that there is no need for protection between applications, leading to some simplification.

Windows CE and Minix 3 are some examples of embedded operating systems.

□ Real-Time Operating Systems

A real-time operating system is a multitasking operating system that aims at executing real-time applications.

□ Real-time systems are usually dedicated, embedded systems.

□ It is a special purpose operating system used when there are rigid time requirements on the operation of a processor or the flow of data,
o thus it is often used as a control device in dedicated applications such as Rocket launching, flight control, robotics, real time simulation, telephone switching equipments, medical imaging systems etc.

o Real time system requires that the processing must be done within the defined time constraints or the system will fail.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 30

CSC224: Operating System I

Real time systems are classified into two categories:

□ Soft Real time System: If certain deadlines are missed then system continues its working with no failure but its performance degrades.

□ Hard Real time System: If any deadline is missed then system will fail to work or does not work properly. This system guarantees that critical task is completed on time.

An operating system does the following activities related to real-time system activity:

- i. The Operating Systems typically read from and react to sensor data.
- ii. The Operating system must guarantee response to events within fixed periods of time to ensure correct performance.

1.5.2 Classification according to the number of users and application they support

□ Single-user, single-tasking - As the name implies, this operating system is designed to manage the computer so that one user can effectively do one thing at a time.

o The Palm OS for Palm handheld computers (e.g. cell phones) is a good example of a modern single-user, single-task operating system.

□ Single-user, multi-tasking - This is the type of operating system most people use on their desktop and laptop computers today.

o Microsoft's Windows and Apple's Mac OS platforms are both examples of operating systems that will let a single user have several

programs in operation at the same time.

o For example, it's entirely possible for a Windows user to be writing a note in a word processor while downloading a file from the Internet while printing the text of an e-mail message.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 31

CSC224: Operating System I

□ Multi-user - A multi-user operating system allows many different users to take advantage of the computer's resources simultaneously.

□ The operating system must make sure that the requirements of the various users are balanced, and that each of the programs they are using has sufficient and separate resources so that a problem with one user doesn't affect the entire community of users.

o Time-sharing system can be classified as multi-user systems as they enable a multiple user access to a computer through the sharing of time.

o UNIX, VMS (Virtual Memory System) and mainframe operating systems, such as MVS (Multiple Virtual Storage), are examples of multi-user operating systems.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 32

CSC224: Operating System I

2. Organization & Design of Operating System

2.1. Operating system functional components

An operating system has many components that manage all the resources in a computer system, ensuring proper execution of programs. We briefly describe these components in this section.

(i) Process manager

A process can be thought of as a program in execution. It needs certain resources, including CPU time, memory, files and I/O devices to accomplish its tasks. The operating system is responsible for:

Creating and terminating both user and system processes

Suspending and resuming processes

Providing mechanisms for process synchronization

Providing mechanisms for process communication

Providing mechanisms for deadlock handling

(ii) Main memory manager

Main memory is a large array of words or bytes (called memory locations), ranging in size from hundreds of thousands to billions.

- o Every word or byte has its own address. Main memory is a repository of quickly accessible data shared by the CPU and I/O devices.

- o It contains the code, data, stack, and other parts of a process. The central processor reads instructions of a process from main memory during the machine cycle4 fetch-decode-execute.

The OS is responsible for the following activities in connection with memory management:

Keeping track of free memory space

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 33

CSC224: Operating System I

Keeping track of which parts of memory are currently being used and by whom

Deciding which processes are to be loaded into memory when memory space becomes available

Deciding how much memory is to be allocated to a process

Allocating and deallocating memory space as needed

Ensuring that a process is not overwritten by another

(iii) Secondary storage manager

The main purpose of a computer system is to execute programs. The programs, along with the data they access, must be in the main memory or primary storage during their execution.

- o Since main memory is too small to accommodate all data and programs, and

- o because the data it holds are lost when the power is lost, the computer system must provide secondary storage to backup main memory.

Most programs are stored on a disk until loaded into the memory and then use disk as both the source and destination of their processing. Like all other resources in a computer system, proper management of disk storage is important.

The operating system is responsible for the following activities in connection with disk management:

Free-space management

Storage allocation and deallocation

Disk scheduling

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 34

CSC224: Operating System I

(iv) I/O system manager

The I/O subsystem consists of:

A memory management component that includes buffering, caching and spooling

A general device-driver interface

Drivers for specific hardware devices

(v) File manager

Computers can store information on several types of physical media, e.g. magnetic tape, magnetic disk and optical disk. The OS maps files onto physical media and accesses these media via the storage devices.

The OS is responsible for the following activities with respect to file management:

Creating and deleting files

Creating and deleting directories

Supporting primitives (operations) for manipulating files and directories

Mapping files onto the secondary storage

Backing up files on stable (non-volatile) storage media

(vi). Protection system manager

If a computer system has multiple users and allows concurrent execution of multiple processes then the various processes must be protected from each other's activities. Protection is any mechanism for controlling the access of programs, processes or users to the resources defined by a computer system.

The protection mechanism must:

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 35

CSC224: Operating System I

Distinguish between authorized and unauthorized usage.

Specify the controls to be imposed.

Provide a means of enforcement.

(vii). Network manager

By creating a network, computers are able to share resources such as CPU, memory etc. There are two ways to configure the operating system to make it network enabled - these are network operating system and distributed operating system.

The operating system is responsible for:

Providing communication system between distributed processors.

Getting information about files/processes/etc. on a remote machine.

Below is a generalized diagram of the operating system organization

Operating System

Processes, Threads, File

Resources Manager Manager

Device

Memory Manager

Manager

Processor(s) Main Devices

Memory

Hardware

A generalized Operating System Organization

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 36

CSC224: Operating System I

2.2 Operating System Design Consideration

As it is the case of any non-trivial software, operating systems tend to be a complex combination of multiple components and many bindings between them. It is almost unthinkable to create such software just as a heap of unstructured code, without any explicit design. Therefore each operating system follows some kind of explicit design that can be captured more formally following design guidelines, principles and rules. In order to design a successful operating system, the designers have to have a clear idea of what they want. Lack of a goal makes it very hard to make subsequent decisions.

2.2.1 Operating System Design Goals

What do operating system designers want? It obviously varies from system to system, being different for embedded systems than for server systems. However, for general-purpose operating systems four main items come to mind:

Define abstractions.

Provide primitive operations.

Ensure isolation.

Manage the hardware.

Each of these items will be discussed below.

□ Define Abstraction: The most important, but probably hardest task of an operating system is to define the right abstractions i.e. hiding unnecessary details to reduce complexity.

o Some of them, such as processes, address spaces, and files, have been around so long that they may seem obvious.

o Others such as threads, are newer, and are less mature. For example, if a multithreaded process that has one thread blocked waiting for keyboard

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 37

CSC224: Operating System I

input forks, is there a thread in the new process also waiting for keyboard input?

- o Other abstractions relate to synchronization, signals, the memory model, modelling of I/O, and many other areas.

- Provision of Primitive Operations: Each of the abstractions can be instantiated in the form of concrete data structures. Users can create processes, files, semaphores, and more.

- o The primitive operations manipulate these data structures. For example, users can read and write files.

- o The primitive operations are implemented in the form of system calls.

- o From the user's point of view, the heart of the operating system is formed by the abstractions and the operations on them available via the system calls.

- Ensure Isolation: Since multiple users can be logged into a computer at the same time, the operating system needs to provide mechanisms to keep them separated.

- o One user may not interfere with another.

- o The process concept is widely used to group resources together for protection purposes.

- o Files and other data structures generally are protected as well.

- o Making sure each user can perform only authorized operations on authorized data is a key goal of system design.

- o However, users also want to share data and resources, so the isolation has to be selective and under user control.

- o This makes it much harder.

- o The e-mail program should not be able to clobber (damage) the Web browser.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 38

CSC224: Operating System I

- o Even when there is only a single user, different processes need to be isolated.

Closely related to this point is the need to isolate failures.

- o If some part of the system goes down, most commonly a user process, it should not be able to take the rest of the system down with it.

- o The system design should make sure that the various parts are well isolated from one another.

- o Ideally, parts of the operating system should also be isolated from one another to allow independent failures.

□ Hardware Management: Finally, the operating system has to manage the hardware..

o In particular, it has to take care of all the low-level chips, such as interrupt controllers and bus controllers.

o It also has to provide a framework for allowing device drivers to manage the larger I/O devices, such as disks, printers, and the display.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 39

CSC224: Operating System I

2.2.2 Operating System Design Principles

(i) Encapsulation Design Principle

The encapsulation principle guides in more detail how the individual components that are assembled into the entire operating system should be structured. The principle states:

- The interaction between components should always be implemented using a well-designed set of interfaces.
- The method of communication should be consistent for any given level of abstraction over the entire operating system.
- No interface and no method of communication should cross multiple levels of abstraction.

The purpose of this design principle is to force the developers to use generic design patterns instead of creating ad hoc solutions.

(ii) Portability Design Principle

The portability design principle guides the implementation decisions of OS, especially with respect to optimizations. The principle states:

- The design and implementation should always maintain a high level of platform neutrality and portability.
- The platform-specific code in the kernel, libraries and tasks should always be clearly separated from the platform-neutral code (either by a complete component decomposition or at least by internal compile-time APIs in the form of a hardware abstraction layer).

This design principle is a guard against optimizing the OS for any given target platform at the cost of making it highly suboptimal on some other platform.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 40

CSC224: Operating System I

o The practical benefit (except for generally more universal design) is to be ready to survive the downfall of the previously dominant hardware platform and to be able to be ported quickly on a new platform.

o Since an operating system represents a major software development effort, it is

desirable to make it easy to port it to different hardware platform.

The need for portability arises when hardware evolves e.g. Windows was first developed for the Intel 80386 platform, which has since evolved into 32 bit Pentium and now 64-bit platforms. Since no one makes 80386 based systems anymore,

□ Microsoft would be out of business if they did not port Windows to these new versions of the Intel CPU architecture!

(iii) Split of Mechanism and Policy Design Principle

The principle states:

- Whenever it is reasonable, each feature of the operating system should be decomposed into two sets of orthogonal aspects 3 one representing the basic mechanism of the feature and one representing the policy on which the mechanism is applied.
- If the mechanism requires it, it can be implemented in a component with a higher criticality level than the component implementing the policy.

A mechanism is a facility that physically enables a feature to be implemented, but does not contain any logic deciding on the conditions when the feature should be applied.

On the other hand, a policy is the logic deciding on the conditions when a feature should be applied, but without the facility that would physically implement the feature. In layman's terms, the mechanism is the <how= and the policy is the <when=.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 41

CSC224: Operating System I

The most important consequence is that the kernel component (as the part of the operating system running on the highest criticality level) should only implement mechanisms and not policies. The policies should be implemented in user space, whenever it is possible.

For example,

- o The timer construct is a mechanism for ensuring CPU protection, but deciding how long the timer is to be set for a particular user is a policy decision.
- o The protection of file through password is a mechanism, while the action to take when password fails is a policy

□ Why is the separation of mechanism and policy desirable?

□ Mechanism and policy must be separate to ensure that systems are easy to modify.

- o No two system installations are the same, so each installation may want to tune the operating system to suit its needs.

□ With mechanism and policy separate, the policy may be changed at will while the mechanism stays unchanged. This arrangement provides a more flexible system.

2.2.3 Why operating System is Hard to Design

(i) Operating systems have become extremely large programs. No one person can sit down at a PC and dash off a serious operating system in a few months. All current versions of UNIX exceed 3 million lines of code; Windows Vista has over 5 million lines of kernel code (and over 70 million lines of total code).

□ No one person can understand 3-5 million lines of code, let alone 70 million lines of code.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 42

CSC224: Operating System I

□ When you have a product that none of the designers can hope to fully understand, it should be no surprise that the results are often far from optimal.

(ii) Operating systems have to deal with concurrency. There are multiple users and multiple I/O devices all active at once.

□ Managing concurrency is inherently much harder than managing a single sequential activity.

(iii) Many users do want to share some of their information and resources with selected other users.

□ The operating system has to make this possible, but in such a way that malicious users cannot interfere.

(iv). Operating system designers really do not have a good idea of how their systems will be used, so they need to provide for considerable generality. Neither UNIX nor Windows was designed with e-mail or Web browsers in mind, yet many computers running these systems do little else.

(v) Modern operating systems are generally designed to be portable, meaning they have to run on multiple hardware platforms. They also have to support thousands of I/O devices, all of which are independently designed with no regard to one another

(vi). The frequent need to be backward compatible with some previous operating system. That system may have restrictions on word lengths, file names, or other aspects that the designers now regard as obsolete, but are stuck with.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 43

CSC224: Operating System I

2.3. Operating System Design Approaches

In this section, we discuss how the operating system components are interconnected and melded into a kernel. Kernel is that part of operating system which directly makes interface with hardware system. Some of its main functions are:

□ To provide a mechanism for creation and deletion of processes

□ To provide processor scheduling, memory management and I/O management

□ To provide mechanism for synchronisation of processes so that processes

synchronize their actions.

□ To provide mechanism for inter-process communication.

Various design approaches have been adopted, each approach producing a unique structure of the operating system. Some of these approaches include

- a. Monolithic e. Modules
- b. Layered f. Hybrid
- c. Microkernel g. Virtual machines
- d. Client 3 Server model

2.3.1 Simple (Monolithic) Structure Approach

In this approach, the entire operating system runs as a single program in kernel mode.

□ The kernel and device drivers are inside one address space running in kernel mode.

□ The operating system is written as a collection of procedures, linked together into a single large executable binary program.

□ When this technique is used, each procedure in the system is free to call any other one, if the latter provides some useful computation that the former needs.

□ In terms of information hiding, there is essentially none4every procedure is visible to every other procedure.

□ Examples include the MS-DOS and the traditional UNIX operating system

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 44

CSC224: Operating System I

Issues with this approach

□ Having thousands of procedures that can call each other without restriction often leads to an unwieldy and difficult to understand the system.

□ Also, it is hard to maintain, patch or extend. Bug fixing or adding new features requires the compilation of the whole source code which is a time and resource consuming process.

In MS-DOS, the interfaces and levels of functionality are not well separated. For instance, application programs are able to access the basic I/O routines to write directly to the display and disk drives.

o Such freedom leaves MS-DOS vulnerable to errant (or malicious) programs, causing entire system crashes when user programs fail.

o Of course, MS-DOS was also limited by the hardware of its era. Because the Intel 8088 for which it was written provides no dual mode and no hardware protection, the designers of MS-DOS had no choice but to leave the base hardware accessible.

Application Programming

Resident System Programming

MS-DOS drivers

ROM 3 BIOS Device Drivers

MS-DOS Structure

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 45

CSC224: Operating System I

The traditional UNIX operating system is layered to some extent.

It consists of two separable parts: the kernel and the system programs.

- o The kernel is further separated into a series of interfaces and device drivers, which have been added and expanded over the years as UNIX has evolved.

As shown in the diagram below, everything below the system-call interface and above the physical hardware is the kernel.

The kernel provides the file system, CPU scheduling, memory management, and other operating-system functions through system calls.

Traditional UNIX System Structure

2.3.2 Layered Structure Approach

The operating system is organized as a hierarchy of layers, each one constructed upon the one below it.

- The operating system is broken into a number of layers (levels).
- The bottom layer (layer 0) is the hardware; the highest (layer N) is the user interface.
- Higher level layer uses the functions (operations) and services of only lower-level layers.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 46

CSC224: Operating System I

The main advantage of the layered approach is the simplicity of construction and debugging. The layers are selected so that each uses functions (operations) and services of only lower-level layers.

- This approach simplifies debugging and system verification.
 - o The first layer can be debugged without any concern for the rest of the system, because, by definition, it uses only the basic hardware (which is assumed correct) to implement its functions.
 - o Once the first layer is debugged, its correct functioning can be assumed while the second layer is debugged, and so on.
- If an error is found during the debugging of a particular layer, the error must be on that layer, because the layers below it are already debugged.
 - o Thus, the design and implementation of the system are simplified.
- Each layer is implemented only with operations provided by lower-level layers.
 - o A layer does not need to know how these operations are implemented; it needs to know only what these operations do.

o Hence, each layer hides the existence of certain data structures, operations, and hardware from higher-level layers.

Below is a representation of a typical layered operating system

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 47

CSC224: Operating System I

2.3.3 Microkernel Structure Approach

This method structures the operating system by removing all nonessential components from the kernel and implementing them as system and user-level programs.

o The result is a smaller kernel.

□ The basic idea behind the microkernel design is to achieve high reliability by splitting the operating system up into small, well-defined modules, only one of which, the microkernel runs in kernel mode and the rest run as relatively powerless ordinary user processes.

□ In particular, by running each device driver and file system as a separate user process, a bug in one of these can crash that component, but cannot crash the entire system

o Thus a bug in the audio driver will cause the sound to be garbled or stop, but will not crash the computer.

o In contrast, in a monolithic system with all the drivers in the kernel, a buggy audio driver can easily reference an invalid memory address and bring the system to a grinding halt instantly.

□ There is little consensus regarding which services should remain in the kernel and which should be implemented in user space.

o Typically, however, microkernel provides minimal process and memory management, in addition to a communication facility.

The following diagram illustrates the architecture of a typical microkernel

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 48

CSC224: Operating System I

Benefits

□ One benefit of the microkernel approach is that it makes extending the operating system easier.

o All new services are added to user space and consequently do not require modification of the kernel.

o When the kernel does have to be modified, the changes tend to be fewer, because the microkernel is a smaller kernel.

o The resulting operating system is easier to port from one hardware design to another.

□ The microkernel also provides more security and reliability, since most

- services are running as user 4 rather than kernel 4 processes.
- o If a service fails, the rest of the operating system remains untouched.

Drawbacks

- Unfortunately, the performance of microkernel can suffer due to increased system-function overhead.
- o Consider the history of Windows NT. The first release had a layered microkernel organization.
- o This version's performance was low compared with that of Windows 95. Windows NT 4.0 partially corrected the performance problem by moving layers from user space to kernel space and integrating them more closely.
- o By the time Windows XP was designed, Windows architecture had become more monolithic than microkernel.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 49

CSC224: Operating System I

2.3.4 Client – Server Model

A slight variation of the microkernel idea is to distinguish two classes of processes,

- o the servers, each of which provides some service, and
- o the clients, which use these services.

This model is known as the client-server model. Often the lowest layer is a microkernel, but that is not required. The essence is the presence of client processes and server processes.

Communication between clients and servers is often by message passing. To obtain a service,

- o a client process constructs a message saying what it wants and sends it to the appropriate service.
- o The service then does the work and sends back the answer.
- o If the client and server happen to run on the same machine, certain optimizations are possible, but conceptually, we are still talking about message passing here.

An obvious generalization of this idea is to have the clients and servers run on different computers, connected by a local or wide-area network, as depicted in the diagram below.

Since clients communicate with servers by sending messages, the clients need not know whether the messages are handled locally on their own machines, or whether they are sent across a network to servers on a remote machine.

The client-server model over a network

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 50

CSC224: Operating System I

2.3.5 Modular Structure Approach

The current methodology for operating-system design involves using loadable kernel modules.

- Here, the kernel has a set of core components and links in additional services via modules, either at boot time or during run time.

- This type of design is common in modern implementations of UNIX, such as Solaris, Linux, and Mac OS X, as well as Windows.

- The idea of the design is for the kernel to provide core services while other services are implemented dynamically, as the kernel is running.

- Linking services dynamically is preferable to adding new features directly to the kernel,

- o which would require recompiling the kernel every time a change was made.

- Thus, for example, we might build CPU scheduling and memory management algorithms directly into the kernel and then add support for different file systems by way of loadable modules.

- The overall result resembles a layered system in that each kernel section has defined, protected interfaces;

- o but it is more flexible than a layered system, because any module can call any other module.

- The approach is also similar to the microkernel approach in that the primary module has only core functions and knowledge of how to load and communicate with other modules;

- o but it is more efficient, because modules do not need to invoke message passing in order to communicate.

The Solaris operating system structure, shown below is organized around a core kernel with seven types of loadable kernel modules:

Scheduling classes Loadable system calls

File systems Executable formats

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 51

CSC224: Operating System I

STREAMS modules Device and bus drivers

Miscellaneous

Solaris Loadable modules

2.3.6 Hybrid Structure Approach

In practice, very few operating systems adopt a single, strictly defined structure.

- o Instead, they combine different structures, resulting in hybrid systems that address performance, security, and usability issues.

- For example, both Linux and Solaris are monolithic, because having the operating

system in a single address space provides very efficient performance.

- o However, they are also modular, so that new functionality can be dynamically added to the kernel.

- Windows is largely monolithic as well (again primarily for performance reasons), but it retains some behaviour typical of microkernel systems, including providing support for separate subsystems (known as operating-system personalities) that run as user-mode processes.

- o Windows systems also provide support for dynamically loadable kernel modules.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 52

CSC224: Operating System I

- Other examples of hybrid systems include the Apple Mac OS X operating system and the two most prominent mobile operating systems 4 iOS and Android.

2.3.7 Virtual machine architecture Approach

- Most computers consist of multiple layers of hardware and software that operate together as a system.

- o Hardware resources typically include a central processor, display, storage, networking, and other peripheral devices.

- o Device drivers installed in the operating system control hardware resources, translating operating-system instructions into the specific device control language.

- o Device drivers assume exclusive device ownership. For example, it is assumed that a video driver owns the video adapter exclusively. Any software application that calls the video adapter must interact with the hardware through the video driver.

- Exclusive device ownership typically precludes running more than one operating system simultaneously on a computer.

- Virtual machine technology overcomes this limitation by redirecting interactions with device resources to lower system levels in such a way that higher-level application layers are unaffected.

- o In a Virtual Machine - each process "seems" to execute on its own processor with its own memory and devices i.e. the operating system creates the illusion of multiple processes, each executing on its own processor with its own (virtual) memory.

- The resources of the physical machine are shared. Virtual devices are sliced out of the physical ones. Virtual disks are subsets of physical ones.

- Useful for running different OS simultaneously on the same machine.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 53

CSC224: Operating System I

□ All virtual machines run within a single process. Each virtual processor runs on its own thread.

o All software code running within the virtual machine runs in a separate VMM (Virtual Machine Monitor) context.

□ The guest operating system and applications run on the virtual machine as if they were running on physical hardware, rather than emulated hardware.

o When a guest operating system is running, the special-purpose VMM kernel manages the CPU and hardware during virtual machine operations, creating an isolated environment in which the guest operating system and applications run close to the hardware at the highest possible performance.

Below is an example of MS-DOS on top of Windows XP.

Java Virtual Machine is another example of virtual machines as shown below

The Java Virtual Machine allows Java code to be portable between various hardware and OS platforms.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 54

CSC224: Operating System I

2.4. Mobile Operating Systems

The two most prominent mobile operating systems are iOS and Android. These operating systems are built on hybrid structures

2.4.1 iOS

The iOS is a mobile operating system designed by Apple to run its smart phone, the iPhone, as well as its tablet computer, the iPad. iOS is structured on the Mac OS X operating system, with added functionality pertinent to mobile devices, but does not directly run Mac OS X applications. The structure of iOS is shown below

Architecture of Apple's iOS

Cocoa Touch is an API for Objective-C that provides several frameworks for developing applications that run on iOS devices. The Cocoa Touch provides support for hardware features unique to mobile devices, such as touch screens.

The media services layer provides services for graphics, audio, and video.

The core services layer provides a variety of features, including support for cloud computing and databases.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 55

CSC224: Operating System I

2.4.2 Android

The Android operating system was designed by the Open Handset Alliance (led primarily by Google) and was developed for Android smart phones and tablet computers.

□ Whereas iOS is designed to run on Apple mobile devices and is close-sourced,

o Android runs on a variety of mobile platforms and is open-sourced, partly explaining its rapid rise in popularity.

□ Android is similar to iOS in that it is a layered stack of software that provides a rich set of frameworks for developing mobile applications.

At the bottom of this software stack is the Linux kernel, although it has been modified by Google and is currently outside the normal distribution of Linux releases. The structure of Android is shown in the diagram below.

Architecture of Google's Android.

□ Linux is used primarily for process, memory, and device-driver support for hardware and has been expanded to include power management.

□ The Android runtime environment includes a core set of libraries as well as the Dalvik virtual machine.

o Software designers for Android devices develop applications in the Java language.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 56

CSC224: Operating System I

o However, rather than using the standard Java API, Google has designed a separate Android API for Java development.

o The Java class files are first compiled to Java bytecode and then translated into an executable file that runs on the Dalvik virtual machine.

□ The Dalvik virtual machine was designed for Android and is optimized for mobile devices with limited memory and CPU processing capabilities.

□ The set of libraries available for Android applications includes

o frameworks for developing web browsers (webkit),

o database support (SQLite), and

o multimedia.

□ The libc library is similar to the standard C library but is much smaller and has been designed for the slower CPUs that characterize mobile devices.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 57

CSC224: Operating System I

3. Processes

3.1. What are Processes?

A Process is an instance of a program in execution. It is a powerful abstraction in Operating System design because:

□ It enables efficient management of running programs ensuring protection of one running program from another.

□ Multiple copies of the same program can be running simultaneously as individual process respectively.

□ It's a building block for building multi-programming and concurrent systems.

From an operating system design perspective, just as you have kernel space and user space, you also have kernel level process(es) running in the kernel space and user level process(es) running in the user space.

A process can use resources such as files in disk storage, input devices, output devices, CPU and so on.

These resources are made available to a process by the operating system

- o The operating system controls these resources
- o In fact in most cases, many processes share the above resources.
- o The operating system is in charge of ensuring optimal usage of these shared resources across processes

Food for thought: Think of the operating system like a government that controls resources of a nation.

A Simplistic View of A Process

When you double click (or tap for touch screens) on your favourite app (let's say firefox for illustration sake), the following happens:

- The operating system reads the program from disk memory (hard disk or memory card in phones),
- prepares it and loads it into main memory (RAM).
- The copy of the program in main memory is what is referred to as a process.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 58

CSC224: Operating System I

- Fun exercise: On windows or linux machine, do the following:
 - Open your favourite app
 - o On Windows, right click on the task bar and click on task manager and then click on Details tab to view running processes.
 - o On Linux (e.g. ubuntu), open system monitor from app menu, and then click on the processes tab.
 - Look through the list of processes and you will see the app you opened.
 - It is worth noting that each process is uniquely identified using a process id

3.2 Process Attributes

Process attributes are information that describes a particular process. Process attribute includes:

1. Identifier 5. Memory Pointers
2. State 6. Context Data
3. Priority 7. I/O Status Information
4. Program Counter 8. Accounting Information

Identifier: Each process is identified by its process identifier, (pid) which is a unique integer.

- o The pid provides a unique value for each process in the system, and it can be used as an index to access various attributes of a process within the kernel.

State: The state of a process is defined in part by the current activity of that process. The state may be new, ready, running, waiting, halted, and so on.

Program counter: The counter indicates the address of the next instruction to be executed for this process

Accounting information: This information includes the amount of CPU and real time used, time limits, account numbers, job or process numbers, and so on.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 59

CSC224: Operating System I

I/O status information: This information includes the list of I/O devices allocated to the process, a list of open files, and so on.

Memory-management information: This information may include such items as the value of the base and limit registers and the page tables, or the segment tables, depending on the memory system used by the operating system

Context Information: It includes the value of the CPU registers, the process and memory- management information

Priority: Represents an approximation of the relative importance of process. Process with a higher priority is placed on the high priority dispatch queue.

3.3 Process State

As a process executes, it changes state. The state of a process is defined in part by the current activity of that process.

Below is a state diagram showing the three states a process may be in:

1. Running (actually using the CPU at that instant).
2. Ready (runnable; temporarily stopped to let another process run).
3. Blocked (unable to run until some external event happens).

Three-state model of a Process

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 60

CSC224: Operating System I

Logically, the first two states are similar. In both cases the process is willing to run, only in the second one, there is temporarily no CPU available for it. The third state is fundamentally different from the first two in that the process cannot run, even if the CPU is idle and has nothing else to do.

Four transitions are possible among these three states, as shown.

□ Transition 1 occurs when the operating system discovers that a process cannot continue right now. In some systems the process can execute a system call, such as pause, to get into blocked state.

o In other systems, including UNIX, when a process reads from a pipe or special file (e.g., a terminal) and there is no input available, the process is automatically blocked.

□ Transitions 2 and 3 are caused by the process scheduler, a part of the operating

system, without the process even knowing about them. Transition 2 occurs when the scheduler decides that the running process has run long enough, and it is time to let another process have some CPU time.

□ Transition 3 occurs when all the other processes have had their fair share and it is time for the first process to get the CPU to run again.

□ Transition 4 occurs when the external event for which a process was waiting (such as the arrival of some input) happens.

o If no other process is running at that instant, transition 3 will be triggered and the process will start running.

o Otherwise it may have to wait in ready state for a little while until the CPU is available and its turn comes.

Using a five-state model, a process may be in one of the following states:

- New. The process is being created.
- Running. Instructions are being executed.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 61

CSC224: Operating System I

- Waiting. The process is waiting for some event to occur (such as an I/O completion or reception of a signal).
- Ready. The process is waiting to be assigned to a processor.
- Terminated. The process has finished execution.

These names are arbitrary, and they vary across operating systems. The states that they represent are found on all systems

o It is important to realize that only one process can be running on any processor at any instant.

o Many processes may be ready and waiting, however.

The state diagram corresponding to these states is presented in the diagram below

Diagram of Process Five-state Model

3.4 Process Control Block (PCB)

Each process is represented in the operating system by a process control block (PCB) also called a task control block.

It contains many pieces of information associated with a specific process. These include process id, state, program counter, CPU registers, memory information and other process attributes discussed earlier.

o the PCB simply serves as the repository for any information that may vary from process to process.

A PCB is shown in the diagram below

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 62

CSC224: Operating System I

PCB is one of the most important data structure/component of an operating system.

The PCB is like a ledger where a <row in the list= contains information about corresponding process. This record is not accessible to the corresponding process or any other process

- o The Operating System stores a list of PCB (one for each process)
- o Thus, each process is unique to the operating system
- o The PCB enables to operating system to manage processes so that there are no chaos
- The difficulty in managing PCBs is not access to it, rather it is the protection of access to it
- o PCBs are read/modified by most modules in the operating system .
- o An error (bug) in one module/component in an operating system can damage the PCBs if accessed/modified incorrectly. This can hinder the operating system from properly managing affected process.
- o A design change in a structure or elements of a PCB can affect several modules/components in the operating system that read/modify PCBs.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 63

CSC224: Operating System I

3.5. Context Switching

Context Switching entails the operating system re-assigning the CPU from one process to another.

- o This is made possible because of the attribute <state= of a process.
- o It enhances efficient utilization of CPU time and other resources
- o It enables concurrency (executing multiple processes)

Switching the CPU to another process requires performing a state save of the current process and a state restore of a different process.

- o When a context switch occurs, the kernel saves the context of the old process in its PCB and loads the saved context of the new process scheduled to run.

Context-switch time is pure overhead, because the system does no useful work while switching.

- o Switching speed varies from machine to machine, depending on the memory speed, the number of registers that must be copied, and the existence of special instructions (such as a single instruction to load or store all registers).

The address space of the current process must be preserved as the space of the next task is prepared for use.

The diagram below shows the switching of CPU between processes

Diagram showing CPU switch from process to process

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 64

CSC224: Operating System I

3.6. Fundamentals of Process Scheduling

Process Scheduling is the steps involved in determining which process in the ready

queue gets assigned the CPU for execution.

In a general sense, apart from CPU resource scheduler, there are different schedulers for different system resources (e.g. file, memory, network and so on), not just CPU resource.

The part of the OS that handles process scheduling is called a Scheduler.

● There are different types of algorithms used to implement a scheduler. Examples include;

3 First Come First Serve (FCFS)

3 Shortest Job First

3 Round Robin (RR)

3 Priority scheduling

- etc

Details about these algorithms will be discussed in CSC312 (Operating System II)

● There is no optimal scheduling algorithm. The resource to allocate, task and other factors determine which scheduling algorithm is suitable for use.

A common representation of process scheduling is a queuing diagram, such as that is shown below

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 65

CSC224: Operating System I

Queuing-diagram representation of process scheduling

Each rectangular box represents a queue. Two types of queues are present: the ready queue and a set of device queues.

The circles represent the resources that serve the queues, and the arrows indicate the flow of processes in the system.

A new process is initially put in the ready queue. It waits there until it is selected for execution, or dispatched. Once the process is allocated the CPU and is executing, one of several events could occur:

- The process could issue an I/O request and then be placed in an I/O queue.
- The process could create a new child process and wait for the child's termination.
- The process could be removed forcibly from the CPU, as a result of an interrupt, and be put back in the ready queue.

In the first two cases, the process eventually switches from the waiting state to the ready state and is then put back in the ready queue.

o A process continues this cycle until it terminates, at which time it is removed from all queues and has its PCB and resources de-allocated.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 66

CSC224: Operating System I

3.7. Operations on Processes

The processes in most systems can execute concurrently, and they may be created and deleted dynamically. Thus, these systems must provide a mechanism for process creation and termination

3.7.1. Process Creation

During the course of execution, a process may create several new processes.

- o The creating process is called a parent process, and
- o the new processes are called the children of that process.
- o Each of these new processes may in turn create other processes, forming a tree of processes.

A process can be created by the OS as a result of different reasons. They include;

- o Spawned by existing process: a user process can initiate a request to create another process. The newly created
- o User Login: when a user logs in, some process(es) are created by the OS
- o OS Service: The OS can create a process to perform a function on behalf a user process (e.g. a process to provide printing service on the computer).

In general, when a process creates a child process, that child process will need certain resources (CPU time, memory, files, I/O devices) to accomplish its task.

- o A child process may be able to obtain its resources directly from the operating system, or it may be constrained to a subset of the resources of the parent process.
- o The parent may have to partition its resources among its children, or it may be able to share some resources (such as memory or files) among several of its children.

When a process creates a new process, two possibilities for execution exist:

- o The parent continues to execute concurrently with its children.
- o The parent waits until some or all of its children have terminated.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 67

CSC224: Operating System I

□ Steps involved in Process Creation

There are different steps involved in creating a process. These include;

- Assign a new process identifier to the new process.
- Allocate space for the process.
- Initialise the Process Control Block (PCB)
- Set the appropriate linkages

Process Creation in Windows OS

□ A process is created in windows using the function:

3 CreateProcess(...)

3 Creates a process from the process which it was called

3 Returns boolean value; true if process (and thread) was created successfully, otherwise false is returned.

3 It does not clone the parent process like unix/linux; rather, you specify the application program to be executed as a parameter

□ Include the header file windows.h in order to use the CreateProcess(...) method.

Process Creation in Unix/Linux OS

To create a process in Unix/Linux, use the function:

□ fork()

□ It creates a clone (child process) of the process (parent process) in which it was called

□ Returns 0 to the calling child process

□ Returns child process id to the parent process

□ If child process could not be created, then fork returns -1

Compiled by: Orhionkpaio, B. C Date: March 2021 Page 68

CSC224: Operating System I

3.7.2 Process Termination

A process terminates when it finishes executing its final statement and asks the operating system to delete it by using the exit() system call.

o At that point, the process may return a status value (typically an integer) to its parent process (via the wait() system call).

o All the resources of the process4including physical and virtual memory, open files, and I/O buffers - are de-allocated by the operating system.

A process can also terminate (end) for other number of reasons. They include;

□ Memory Unavailable: process requires more memory than the current available space in memory.

□ Bounds Violation: process tries to access memory location/region that it is not allowed to access (e.g. memory address space of another process, memory location of OS kernel and so on).

□ Protection Error: process tries to use a resource (e.g. file) that it is not allowed to use or use the resource inappropriately (e.g. writing to a read-only file).

□ Parent Process Termination: only affects processes created by another process. When the parent process terminates, the OS may terminate all child processes of the parent process.

□ User or OS Intervention: User exit the program or the OS forcefully terminates the process (for example, to resolve resource deadlock).

□ I/O Failure: Failure of an I/O device being used by process.

□ Invalid Instruction: process attempts to execute an invalid instruction (e.g. branching into data segment and then attempting to execute the data).

Compiled by: Orhionkpaio, B. C Date: March 2021 Page 69

CSC224: Operating System I

3.8 Fundamentals of Inter-Process Communication (IPC)

Inter-Process Communication (IPC) is the mechanism set up in place to enable two processes communicate with each other.

- Fundamental requirements for IPC that need to be satisfied;

3 Synchronisation: to ensure information sent by sender process gets to the receiver process.

3 Communication: exchange of information.

Processes executing concurrently in the operating system may be either independent processes or cooperating processes.

- o A process is independent if it cannot affect or be affected by the other processes executing in the system. Any process that does not share data with any other process is independent.

- o A process is cooperating if it can affect or be affected by the other processes executing in the system.

- o Clearly, any process that shares data with other processes is a cooperating process..

There are several reasons for providing an environment that allows process cooperation:

- Information sharing:.. Since several users may be interested in the same piece of information (for instance, a shared file), we must provide an environment to allow concurrent access to such information

- Computation speedup:.. If we want a particular task to run faster, we must break it into subtasks, each of which will be executing in parallel with the others. Notice that such a speedup can be achieved only if the computer has multiple processing cores.

- Modularity: We may want to construct the system in a modular fashion, dividing the system functions into separate processes or threads.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 70

CSC224: Operating System I

- Convenience: Even an individual user may work on many tasks at the same time. For instance, a user may be editing, listening to music, and compiling in parallel.

Cooperating processes require an interprocess communication (IPC) mechanism that will allow them to exchange data and information.

There are two fundamental models of interprocess communication: shared memory and message passing.

- In the shared-memory model, a region of memory that is shared by cooperating processes is established. Processes can then exchange information by reading and writing data to the shared region.

- In the message-passing model, communication takes place by means of

messages exchanged between the cooperating processes.

- Message passing is useful for exchanging smaller amounts of data, because no conflicts need be avoided.
- Message passing is also easier to implement in a distributed system than shared memory.
- Shared memory can be faster than message passing, since message-passing systems are typically implemented using system calls

Communications models. (a) Message passing. (b) Shared memory

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 71

CSC224: Operating System I

3.8.1. Shared-Memory Systems

Inter-process communication using shared memory requires communicating processes to establish a region of shared memory.

Typically, a shared-memory region resides in the address space of the process creating the shared-memory segment.

Other processes that wish to communicate using this shared-memory segment must attach it to their address space.

- o Recall that, normally, the operating system tries to prevent one process from accessing another process's memory.
- o Shared memory requires that two or more processes agree to remove this restriction.
- o They can then exchange information by reading and writing data in the shared areas.
- o The processes are also responsible for ensuring that they are not writing to the same location simultaneously

3.8.2 Message-Passing Systems

Message passing provides a mechanism to allow processes to communicate and to synchronize their actions without sharing the same address space.

- o It is particularly useful in a distributed environment, where the communicating processes may reside on different computers connected by a network.
- o For example, an Internet chat program could be designed so that chat participants communicate with one another by exchanging messages.

A message-passing facility provides at least two operations:

send(destination, message) receive(Source, message)

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 72

CSC224: Operating System I

A process sends a message to another process (specified as destination parameter) by calling send(...).

While the receiving process receives the message from the sender (specified as

source) by calling receive(...).

If processes P and Q want to communicate, they must send messages to and receive messages from each other: a communication link must exist between them.

This link can be implemented in a variety of ways

- Direct or indirect communication
- Synchronous or asynchronous communication
- Automatic or explicit buffering

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 73

CSC224: Operating System I

3.8.3 Communication in Client/Server Systems

Client/Server paradigm is a computer networking concept where a client (computer) makes a request to access some form of resource on a server (computer) over a Computer Network.

The resource on the server could be files, CPU, memory and so on.

● Example of servers include;

3 File server

3 Web server

3 Mail Server

3 and so on

● One of the most popular client/server system is the web/internet.

A user access a website (e.g. <nairaland.com=>) from a client computer using web browser software (e.g. Opera) by making a request to an online server computer running a web server program.

□ The user's web browser is a process on the client computer. Also, the web server program in the server computer is a process.

□ Making a request means the client process sends a message (over the internet) to the server process.

□ The message is sent indirectly through mail box (port) on the server computer.

□ The server reads the message in the port, executes some instructions and sends a message (a web page) back to the client over the internet.

□ The message reaches the client's computer and it is stored in a mail box. The client process (web browser) reads the message and displays the web page on screen.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 74

CSC224: Operating System I

4. Threads

4.1 What are Threads?

A thread is a unit of execution within a process. Hence a single process can contain multiple threads, each carrying out different tasks.

You can consider a thread as a light weight process.

- Light weight because creation/management of threads does not require expensive operations when compared to process creation.
- All threads in a process share the same memory address space of the process to which it belongs. This is different from the process abstraction where each process has its own memory address space (even when the processes belong to the same application).
- In addition to sharing the memory address space to which a thread belongs, each thread also have their own unique allocation to store local thread variables and other information.

A traditional (or heavyweight) process has a single thread of control.

- o If a process has multiple threads of control, it can perform more than one task at a time.
- o Most software applications that run on modern computers are multithreaded.

A web browser might have one thread display images or text while another thread retrieves data from the network, for example.

- o A busy web server may have several (perhaps thousands of) clients concurrently accessing it. If the web server ran as a traditional single-threaded process, it would be able to service only one client at a time,
- o a client might have to wait a very long time for its request to be serviced.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 75

CSC224: Operating System I

A word processor may have a thread for displaying graphics, another thread for responding to keystrokes from the user, and a third thread for performing spelling and grammar checking in the background.

- An example of word processor with three threads (one each for spell checking, keyboard, disk backup) is shown in the diagram below

A word processor with three threads

4.2 Types of Threads

There are two major types of Threads

- User Level Thread
- Kernel Level Thread

Another type of thread is the combination of concepts from User Level Thread and Kernel Level Thread.

- Combined approach

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 76

CSC224: Operating System I

4.2.1 User Level Threads (ULTs)

Thread management is carried out by application running in the User Space of the

OS..

o ULTs provide a Library of functions to allow user processes to create and manage their own threads,

- Kernel is not aware of existence of thread.

- User threads are supported above the kernel and are managed without kernel support,

- Creation of new threads, switching, and synchronization are done without the intervention of the kernel

- Context switching for ULTs does not require the kernel space code execution.

- This saves time and allows context switching to be faster for ULTs than processes.

- Main advantage is the speed of execution of operations to manage thread. This is because it does not require any kernel space operation.

- Kernel space operations are expensive.

- The main drawback is that

- o when a thread is in a blocked state, then the process to which it belongs (and the other threads in the process) are placed in a blocked state.

- o Also, it does not thrive in multi-threaded processors since no more than one thread can be scheduled at the same time

Examples include Solaris 1 & 2, NT thread etc

The diagram below shows user level thread a

A User level Thread

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 77

CSC224: Operating System I

4.2.2 Kernel Level Threads (KLTs)

Threads within a process are managed by the OS kernel (in kernel space). The kernel provides system calls to create and manage threads.

- kernel threads are supported and managed directly by the operating system.

- The kernel has full knowledge of all threads

- Virtually all contemporary operating systems including Windows, Linux, Mac OS X, and Solaris support kernel threads.

- Context switching between threads will require kernel level operations.

- Advantage is that a thread in blocked state does not block the entire process to which it belongs.

- Main drawback is that since it requires kernel level operations, it means that its management is slower than that of ULTs.

- Kernel space operations are more expensive to process than user space operations.

- Increased in complexity since the kernel must manage and schedule threads

Examples include BSD, Solaris 2, Windows 95 and NT etc

Kernel Level Threads

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 78

CSC224: Operating System I

4.2.3 Combined Approach

This approach combines the ULTs and KLTs approaches.

- o Thread creation is carried out in the user space,
- o while other thread management activities are carried out in the kernel space.

Combined Approach

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 79

CSC224: Operating System I

4.3. Multi-Threading Models

A relationship exists between user threads and kernel threads. In this section, we look at three common ways of establishing such a relationship: the many-to-one model, the one-to-one model, and the many-to many models

4.3.1 Many-to-One Model

The many-to-one model (see diagram below) maps many user-level threads to one kernel thread.

- Thread management is done by the thread library in user space, so it is efficient.
 - However, the entire process will block if a thread makes a blocking system call.
 - Also, because only one thread can access the kernel at a time, multiple threads are unable to run in parallel on multicore systems.
- Green threads^{4a} a thread library available for Solaris systems and adopted in early versions of Java⁴ used the many-to-one model.
- However, very few systems continue to use the model because of its inability to take advantage of multiple processing cores.

Many-to-one Model

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 80

CSC224: Operating System I

4.3.2 One-to-One Model

The one-to-one model maps each user thread to a kernel thread.

It provides more concurrency than the many-to-one model

- o by allowing another thread to run when a thread makes a blocking system call.
- o It also allows multiple threads to run in parallel on multiprocessors.

The only drawback to this model is that

- o creating a user thread requires creating the corresponding kernel thread.
- o Because the overhead of creating kernel threads can burden the performance of an application, most implementations of this model restrict the number of

threads supported by the system.

Linux, along with the family of Windows operating systems, implement the one-to-one model.

One-to-One Model

4.3.3. Many-to-Many Model

The many-to-many model multiplexes many user-level threads to a smaller or equal number of kernel threads.

- The number of kernel threads may be specific to either a particular application or a particular machine (an application may be allocated more kernel threads on a multiprocessor than on a single processor).

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 81

CSC224: Operating System I

Many-to-Many Model

One variation on the many-to-many model still multiplexes many user level threads to a smaller or equal number of kernel threads but also allows a user-level thread to be bound to a kernel thread.

- This variation is sometimes referred to as the two-level model
- The Solaris operating system supported the two-level model in versions older than Solaris 9.
- However, beginning with Solaris 9, this system uses the one-to-one model.

Two-Level Model

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 82

CSC224: Operating System I

4.4 Benefit of Threads

Whether ULTs or KLTs, the following are some benefit of threads over processes:

- Takes less time to create a new thread than a new process.
- Takes less time to terminate a thread than a process.
- Context switching between two threads takes less time than between two processes.
- Threads also play a vital role in remote procedure call (RPC) systems.
 - o When a server receives a message, it services the message using a separate thread.
 - o This allows the server to service several concurrent requests.

4.5 Issues with Threads

- Even though threads share address space of the process to which they belong, each thread contains a unique storage allocation for stack memory (for its own function call operations and local variables). This can cause problems as stated below.
 - o This additional memory may not necessarily be protected from

other threads in the process.

- o Hence the programmer has to ensure that threads only read-from or write-to the memory location that it is allocated (unless when there is need to share data)

- o Recall that memory allocated to a process is protected from other process by default.

- The idea of multiple threads (of same process) sharing its process data/variables is an advantage and a disadvantage

- Advantage because you don't need communication between threads (which is an added computation overhead as seen in inter-process communication) in order to share data.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 83

CSC224: Operating System I

- Disadvantage because allowing multiple threads to access same data/variable at an instance in time can lead to corruption of data which leads to data inconsistency or could possibly crash your program.

- o This is called a Race Condition

To summarise, if not properly managed, possible problems that can arise from using threads include:

- Lack of protection of unique memory address space allocated to thread.

Threads belonging to same process can write/read from each other's memory address space.

- Lack of protection of process memory address space to which threads belongs. All threads belonging to a process have equal access to the process's memory address space and can corrupt stored data as a result of concurrent execution of thread.

4.5.1 Threads and Race Conditions

A Race Condition occurs when two or more threads of a process read-from and write-to a process memory address space (e.g. a variable declared inside the process).

For demonstration purpose, consider the example below.

- Let's say we want a program to multiply a variable (called wage with current value as 50) by 2 and then add 100 to the result and then store the final result in the same variable. We can write the program to contain 2 threads. One thread for the multiplication task and the other for the addition task. If the program runs correctly, the final answer should be 200. However, there are different orders that the threads can run concurrently.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 84

CSC224: Operating System I

- If the threads are executed concurrently (without being programmatically

careful), a possible execution path that can lead to corrupting the data stored in process variable wage is given below:

3 Thread 1 reads the current value of wage (i.e. 50)

3 Thread 2 reads the current value of wage. (i.e. 50)

3 Thread 1 multiplies the read value by 2 (i.e. $50 \times 2 = 100$)

3 Thread 2 adds 100 to the read value (i.e. $50 + 100 = 150$)

3 Thread 1 writes result back to wage. variable (i.e. set wage to 100)

3 Thread 2 writes result back to wage. variable (i.e. set wage to 150)

□ The result from the example (in the previous slide) is incorrect.

o The reason is because the two threads are simultaneously reading from and write to the shared process variable wage.

o For the program to work correctly, each thread 2 has to wait for thread 1 to finish its read from and writing to variable wage.

o This leads to the concept of <critical code section=.

4.5.2 Critical Code Section

A critical code section in a Multi-Thread program is a section of the program code that requires threads to be executed sequentially rather than concurrently.

□ This occurs (as seen in the example) when a thread tries to use a process variable that another thread is reading from or writing-to.

The concept of critical code section can be generalised broadly to computer resources and not just variables.

□ If a thread contains a program section that uses a resource (e.g. network, file read/write, process variable and so on) required by other threads, then that program section of thread is a critical code section.

The OS concept that ensures sequential execution of critical code section among threads (of a process) is called Thread Synchronisation.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 85

CSC224: Operating System I

4.5.3 Thread Synchronisation

In order to prevent race conditions in critical code sections, we need to employ synchronisation techniques.

□ Synchronization meaning that when threads get to a critical code section, then they are executed sequentially (one after the other) and not concurrently (not parallel)

● Synchronisation techniques include:

3 Mutex

3 Semaphore

3 Condition Variables

● The code for thread synchronisation is written by the programmer inside the Multi-Threaded program. This is why it was earlier stated that a programmer have to be

careful when using threads. The program is tasked with the responsibility to:

3 Identify the critical code sections of the programs.

3 Implement a thread synchronisation strategy/technique to combat race conditions.

Details on synchronization are discussed in CSC312 (Operating System II)

4.6 Processes vs Threads

Process Thread

Typically independent Exist as sub set of process

Have separate address space Share their address space

Context switch involves kernel and thus Faster context switch which is performed by slows down the process a user level library

Interacts through IPC (Inter-Process No need for IPC since threads can share Communication) only common data

Protected No protection between threads. It is quite possible that one thread overwrites the stack of another thread

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 86

Paper 5 of 5 — CSC 224 operating system

Level 200 • Second Semester • Handout

CSC224: Operating System I

Department of Computer Science

Federal University of Petroleum Resources, Effurun,

Delta State

OPERATING SYSTEM I

[CSC224]

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 1

CSC224: Operating System I

COURSE GOAL: To provide the students with the fundamental knowledge of operating system necessary to prepare them for future courses in operating system and to apply them in real world situations.

BEHAVIOURAL OBJECTIVES:

At the end of the course, students should be able to:

- i. Define Operating System and discuss its functions
- ii. Discuss the hardware support for operating system
- iii. Describe how the operating system operates
- iv. Categorize Operating System
- v. List and explain the major components of Operating System
- vi. Describe the services an Operating System provides to users, processes and other systems
- vii. Discuss the various ways of structuring an Operating System
- viii. Enumerate and discuss the design consideration of operating system
- ix. Discuss the design goals and architecture of some operating systems
- x. Explain why Operating System are sometimes difficult to design
- xi. Describe the various features of processes, including scheduling, creation, and termination
- xii. Describe communication among processes
- xiii. Explain the concept of race condition and critical sections in Processes/ threads
- xiv. Examine some strategies that provide implicit threading
- xv. Explain the benefits and challenging issues of Threads
- xvi. Differentiate between processes and threads

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 2

CSC224: Operating System I

COURSE OUTLINE

1. Meaning, functions, and support for Operating System

- i. What is Operating system?
- ii. Operating system services
- iii. Hardware support for operating systems
- iv. Execution of the Operating system
- v. Classification of operating system
- 2. Organization & Design of Operating System
 - i. Operating system functional components
 - ii. Operating system design consideration
 - iii. Operating system design Approaches
 - a. Monolithic e. Modules
 - b. Layered f. Hybrid
 - c. Microkernel g. Virtual machines
 - d. Client – Server model
 - iv.. Mobile Operating Systems
- 3. Processes
 - i. What are Processes? vi. Process Creation and
 - ii. Process Attributes Termination
 - iii. Process Control Block vii. Fundamentals of Inter-
 - iv. Process State Models Process Communication
 - v. Fundamentals of Process viii. Communication in client Scheduling – server systems
- 4. Threads
 - i. Types of Threads iv. Issues with Threads
 - ii. Multi-Threading Models v. Processes Vs Threads
 - iii. Benefits of Threads

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 3

CSC224: Operating System I

LECTURE SCHEDULE

Week Topic Sub-topics

- 1 Course Orientation Syllabus and Policies
- 2-3 Meaning, functions, and i. What is Operating system? support for Operating System ii. Operating system services
- iii. Hardware support for operating systems
- iv. Execution of the Operating system
- v. Classification of operating system
- 4-6 Organization & Design of i. Operating system functional components Operating System ii. Operating system design consideration
- iii. Operating system design Approaches
- iv. Mobile Operating System

- 7-9 Processes i. What are Processes?
- ii. Process Attributes
- iii. Process Control Block
- iv. Process State Models
- v. Fundamentals of Process Scheduling
- vi. Process Creation and Termination
- vii. Fundamentals of Inter-Process Communication
- viii. Communication in client – server systems
- 10 - 11 Threads i. Types of Threads
- ii. Multi-Threading Models
- iii. Benefits of Threads
- iv. Issues with Threads
- v. Processes Vs Threads
- 12 Revision

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 4

CSC224: Operating System I

TEXTBOOK

Operating System Concepts, Abraham Silberchatz, Peter B. Galvin, Greg Gagne, Ninth Edition, 2012, Wiley.

Modern Operating Systems, Andrew S. Tanenbaum, Herbert Bos Pearson Education, Inc Fourth Edition 2008

CONTACTS:

GRADING

1. CA (30mks):

- (i) Assignments-----12mks (or 0 for copying)
- (ii) Attendance.....3mks (70% total attendance)
- (iii) Participation.....3mks
- (iv) Test-----12mks (or 0 for cheating)

2. Exam (70mks)

POLICIES

(i) Attendance - It is expected that every student will be in class for lectures. Attendance records will be kept and used to award marks for each student. Only students with up to or above 75% attendance will be awarded 3mks. Signing for friends is an act of DISHONESTY and it is HIGHLY PROHIBITED.

(ii) Makeup test - There shall be no makeup test. The date for the test shall be announced to the class at least one week before the test.

(iii) Academic Integrity – Any form of academic dishonesty, such as copying during test /exam or making copies of other students' work, is prohibited.

(iv). Submission of Assignments/Lab work – Late submission of assignment or lab work will be graded as Score – x, where x is the number of days of late submission..

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 5

CSC224: Operating System I

1. Meaning, functions, and support for Operating System

1.1 What is Operating System?

An Operating System (OS) is a software that manages the computer hardware, as well as providing an environment for application programs to run.

□ That is the Operating System acts as an interface between a user of the computer and the computer hardware and also provides an environment in which we can execute programs.

The purpose of an operating system involves two key goals:

- Availability of a convenient, easy-to-use, and powerful set of services that are provided to the users and the application programs in the computer system
- Management of the computer resources in the most efficient manner

Operating systems are virtually present in almost everything we use today including toasters, cars, ships, spacecraft.. They are the basis for game machines, music players, cable TV tuners, and industrial control systems. A computer system has many hardware and software resources that may be required to solve a problem: CPU time, memory space, files storage space, I/O devices etc.

□ The operating system acts as the manager of these resources

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 6

CSC224: Operating System I

1.2 Operating System Services

An Operating System provides services to both the users and to the programs.

- It provides programs an environment to execute.
- It provides users the services to execute the programs in a convenient manner.

Following are a few common services provided by an operating system:

- i. User interface vi. Error Detection
- ii. Program execution vii. Resource Allocation
- iii. I/O operations viii. Protection
- iv. File System manipulation ix. Accounting
- v. Communication

(i) User interface: Almost all operating systems have a user interface (UI).

This interface can take several forms.

- One is a command-line interface (CLI), which uses text commands and a method for entering them (say, a keyboard for typing in commands in a specific format with specific options).
- Another is a batch interface, in which commands and directives to control those commands are entered into files, and those files are executed.
- Most commonly, a graphical user interface (GUI) is used. Here, the interface is a window system with a pointing device to direct I/O, choose from menus, and make selections and a keyboard to enter text.
- Some systems provide two or all three of these variations.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 7

CSC224: Operating System I

(ii) Program Execution

Operating systems handle many kinds of activities from user programs to system programs like printer spooler, name servers, file server, etc.

- Each of these activities is encapsulated as a process.
- A program that is loaded into memory and is executing is commonly referred to as a process.□

The following are the major activities of an operating system with respect to program management:

- Loads a program into memory
- Handles program's execution
- Provides a mechanism for process synchronization
- Provides a mechanism for process communication
- Provides a mechanism for deadlock handling

(iii) I/O Operation

An I/O subsystem comprises of I/O devices and their corresponding driver software.

Drivers hide the peculiarities of specific hardware devices from the users.

An Operating System manages the communication between user and device drivers.

- I/O operation means read or write operation with any file or any specific I/O device.
- Operating system provides the access to the required I/O device when required.

(iv). File System Manipulation

A file represents a collection of related information. Computers can store files on the disk (secondary storage), for long-term storage purpose.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 8

CSC224: Operating System I

- o Examples of storage media include magnetic tape, magnetic disk and optical

disk drives like CD, DVD.

- o Each of these media has its own properties like speed, capacity, data transfer rate and data access methods.

A file system is normally organized into directories/folders for easy navigation and usage. These directories/folders may contain files and other directories/folders.

The following are the major activities of an operating system with respect to file management:

- Program needs to read a file or write a file.

- o The operating system gives the permission to the program for operation on file.

- o Permission varies from read-only, read-write, denied, and so on.

- Operating System provides an interface to the user to create/delete files.

- Operating System provides an interface to the user to create/delete directories/folders.

- Operating System provides an interface to create the backup of file system.

(v) Communication

In case of distributed systems which are a collection of processors that do not share memory, peripheral devices, or a clock, the operating system manages communications between all the processes. Multiple processes communicate with one another through communication lines in the network.

The following are the major activities of an operating system with respect to communication:

- Two processes often require data to be transferred between them.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 9

CSC224: Operating System I

- o Both the processes can be on one computer or on different computers, but are connected through a computer network.

- Communication may be implemented by two methods, either by Shared Memory or by Message Passing.

- The operating system manages communications between all the processes

(vi). Error Handling

Errors can occur anytime and anywhere. An error may occur in CPU, in I/O devices or in the memory hardware.

The following are the major activities of an operating system with respect to error handling:

- The OS constantly checks for possible errors.

- The OS takes an appropriate action to ensure correct and consistent computing.

(vii). Resource Management

In case of multi-user or multi-tasking environment, resources such as main memory, CPU cycles and files storage are to be allocated to each user or job.

The following are the major activities of an operating system with respect to resource management:

Manage secondary storage devices

Allocate appropriate amount of disk space when files are created

Deallocate space when files are removed

Ensure that a new file does not overwrite an existing file Schedule disk requests

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 10

CSC224: Operating System I

Manage primary storage

Allocate appropriate amount of memory space when programs are to be loaded into the memory for execution

Deallocate space when processes terminate

Ensure that a new process is not loaded on top of an existing process

Ensure that a process does not access memory space that does not belong to it Minimize the amount of unused memory space

Allow execution of programs larger in size than the available main memory

Manage processes

Allow simultaneous execution of processes by scheduling the CPU(s)

Prevent deadlocks between processes

Ensure integrity of shared data

Synchronize executions of cooperating processes

(viii). Protection

Considering a computer system having multiple users and concurrent execution of multiple processes, the various processes must be protected from each other's activities.

□ Protection refers to a mechanism or a way to control the access of programs, processes, or users to the resources defined by a computer system.

The following are the major activities of an operating system with respect to protection:

□ The OS ensures that all access to system resources is controlled.

□ The OS ensures that external I/O devices are protected from invalid access attempts.

□ The OS provides authentication features for each user by means of passwords.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 11

CSC224: Operating System I

(ix) Accounting. An operating system collects usage statistics for various resources and monitor performance parameters such as response time.

o On any system, this information is useful in anticipating the need for future enhancements and in tuning the system to improve performance.

o On a multiuser system, the information can be used for billing purposes.

1.3. Hardware Support for Operating Systems

Modern computers are built on the Von Neumann architecture. The Von Neumann architecture uses a single processor which follows a linear sequence of fetch-decode-execute.

□ In order to do this, the processor has to use some special registers, which are discrete memory locations with special purposes attached.

The following table shows these registers and their functions:

Register Function

Program Counter (PC)

Keeps track of where to find the next instruction so that a copy of the instruction can be placed in the current instruction register. Sometimes the program counter is called the Sequence Control Register (SCR) as it controls the sequence in which instructions are executed.

Current Instruction Register (CIR) Holds the instruction that is to be executed

Memory Address Register (MAR) Used to hold the memory address that contains either the next piece of data or an instruction that is to be used.

Memory Data Register (MDR) Acts like a buffer and holds anything that is copied from the memory ready for the processor to use it.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 12

CSC224: Operating System I

Index Register (IR) Used for modifying operand addresses during the run of a program, typically for doing vector/array operations.

Accumulator (ACC) Holds results

The following diagram shows the layout of the registers in the CPU and their data paths

Memory

CPU

Control Unit ALU

PC Accumulator

CIR

MAR

MDR

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 13

CSC224: Operating System I

1.4. Executing the Operating System

1.4.1 Operating System Modes

A typical operating system has at least two (dual) modes of operation: user mode and kernel mode (also called supervisor mode, system mode, or privileged mode).

□ A bit, called the mode bit (a bit in a special register called the processor status word - PSW), is added to the hardware of the computer to indicate the current mode: kernel (0) or user (1).

□ User applications run in user mode.

The heart of the operating system is called the kernel and provides a collection of functions that perform the basic services such as scheduling applications.

- o The kernel runs in kernel mode.

- o At system boot time, the hardware starts in kernel mode.

- o The operating system is then loaded and starts user applications in user mode.

Various CPU instructions are only available to software running in kernel mode, i.e., when the bit is set. Hence these privileged instructions can only be executed by the operating system, and not by user applications.

Examples privileged instructions include:

- Instructions to set the interrupt priority level (IPL). This can be used to block certain classes of interrupts from occurring, thus guaranteeing undisturbed execution.
- Instructions to set the hardware clock to generate an interrupt at a certain time in the future.
- Instructions to activate I/O devices. These are used to implement I/O operations on files.
- Instructions to load and store special CPU registers, such as those used to define the accessible memory addresses, and the mapping from each application's virtual addresses to the appropriate addresses in the physical memory.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 14

CSC224: Operating System I

- Instructions to load and store values from memory directly, without going

through the usual mapping. This allows the operating system to access all the memory.

The dual mode of operation provides us with the means for protecting the operating system from errant users — and errant users from one another.

- o This protection is accomplished by designating some of the machine instructions that may cause harm as privileged instructions, such as those listed above.

The hardware allows privileged instructions to be executed only in kernel mode.

- If an attempt is made to execute a privileged instruction in user mode, the hardware does not execute the instruction but rather treats it as illegal and traps it to the operating system.

The lack of a hardware-supported dual mode can cause serious shortcomings in an operating system.

- o For instance, MS-DOS was written for the Intel 8088 architecture, which has no mode bit and therefore no dual mode.

- o A user program running awry (improperly) can wipe out the operating system by writing over it with data; and multiple programs are able to write to a device at the same time, with potentially disastrous results.

Modern versions of the Intel CPU do provide dual-mode operation. Accordingly, most contemporary operating systems — such as Microsoft Windows 7, as well as Unix and Linux — take advantage of this dual-mode feature and provide greater protection for the operating system.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 15

CSC224: Operating System I

- How to Enter the Kernel Mode:

There are two ways to enter kernel mode: interrupts and system calls.

(a) Interrupts: Interrupts are special conditions that cause the CPU not to execute the next instruction.

- Instead, it enters kernel mode and executes an operating system interrupt handler.

- Modern operating systems are interrupt driven.

But how does the CPU (hardware) know the address of the appropriate kernel function?

- When the system is booted, the operating system stores the addresses of the interrupt handling functions in the interrupt vector.

- When an interrupt occurs, the hardware stores the current PSW (Processor Status Word) and PC and loads the appropriate PSW and PC values for the interrupt handler.

- o The PC (program counter) is a special register that holds the address

of the next instruction to be executed.

- o The PSW indicates execution in kernel mode.

- The PC is obtained by using the interrupt number as an index into the interrupt vector, and using the address found there.

- Note that the hardware does this blindly using the predefined address of the interrupt vector as a base.

- The interrupt vector is a predefined location in memory where all the entry points to the operating system are stored.

- It is indexed by interrupt number: the address of the handler for interrupt *i* is stored in entry *i* of the vector.

- When a certain interrupt occurs, the hardware uses the interrupt number as an index into the interrupt vector, and loads the value found there into the PC, thus jumping to the appropriate handler.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 16

CSC224: Operating System I

- o It is up to the operating system to actually store the correct addresses in the correct places.

- o If it does not, this is a bug in the operating system.

- Types of Interrupts

There are two main types of interrupts: external (asynchronous) and internal (synchronous).

i. External (Asynchronous) interrupts are generated by external devices at unpredictable times. Examples include:

- o Clock interrupt: This tells the operating system that a certain amount of time has passed.

- It's handler is the operating system function that keeps track of time. S

- Sometimes, this function also calls the scheduler which might pre-empt the current application and run another in its place.

- Without clock interrupts, the application might run forever and monopolize the computer.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 17

CSC224: Operating System I

- o I/O device interrupt: This tells the operating system that an I/O operation has completed. The operating system then wakes up the application that requested the I/O operation.

ii. Internal (synchronous) interrupts occur as a result of an exception condition when executing the current instruction (as this is a result of what the software did, this is sometimes also called a "software interrupt"). This means that the processor

cannot complete the current instruction for some reason, so it transfers responsibility to the operating system. There are two main types of exceptions:

- o An error condition: This tells the operating system that the current application did something illegal (division by zero, arithmetic overflow, trying to issue a privileged instruction, etc.). The handler is the operating system function that deals with misbehaved applications; usually, it kills them.
- o A temporary problem: for example, the process tried to access a page of memory that is not allocated at the moment. This is an error condition that the operating system can handle, and it does so by bringing the required page into memory.

When the handler finishes its execution, the execution of the interrupted application continues where it left off — except if the operating system killed the application or decided to schedule another one.

(b) System calls: The second way to enter kernel mode is the system calls. An application can also explicitly transfer control to the operating system by performing a system call.

□ System calls provide the means for a user program to ask the operating system to perform tasks reserved for the operating system on the user program's behalf.

□ This is implemented by issuing the trap instruction.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 18

CSC224: Operating System I

- o This instruction causes the CPU to enter kernel mode, and set the program counter to a special operating system entry point.

The operating system then performs some service on behalf of the application.

Technically, this is actually just another (internal) interrupt — but a desirable one that was generated by an explicit request.

Typical system calls include:

- Open, close, read, or write to a file.
- Create a new process (that is, starts running another application).
- Get some information from the system, e.g. the time of day.
- Request to change the status of the application, e.g. to reduce its priority or to allow it to use more memory.

As an operating system can have more than a hundred system calls, the hardware cannot be expected to know about all of them (as opposed to interrupts, which are a hardware thing to begin with).

The sequence of events leading to the execution of a system call is therefore slightly more involved:

- i. The application calls a library function that serves as a wrapper for the system call.
- ii. The library function (still running in user mode) stores the system call identifier and the provided arguments in a designated place in memory.
- iii. It then issues the trap instruction.
- iv. The hardware switches to privileged mode and loads the PC with the address of the operating system function that serves as an entry point for system calls.
- v. The entry point function starts running (in kernel mode). It looks in the designated place to find which system call is requested.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 19

CSC224: Operating System I

- vi. The system call identifier is used in a big switch statement to find and call the appropriate operating system function to actually perform the desired service. This function starts by retrieving its arguments from where they were stored by the wrapper library function.

The diagram below shows the transition from user to kernel mode due to user application request.

1.4.2 Starting an Operating System/ Booting the Computer

On the Parent board (i.e. motherboard).is a program called the system BIOS (Basic Input Output System). – see diagram below

□ The BIOS contains low-level I/O software, including procedures to read the keyboard, write to the screen, and do disk I/O, among other things.

When the computer is powered on, the BIOS is started.

- o It first checks to see how much RAM is installed and whether the keyboard and other basic devices are installed and responding correctly.
- o It starts out by scanning the ISA and PCI buses to detect all the devices attached to them.
- o This process is commonly referred to as Power-On Self Test (POST).

At this stage the computer's CPU is operating in Real Mode (or real address mode),

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 20

CSC224: Operating System I

- o which means that it is only capable of using approximately 1 MB of memory built into the Parentboard

A sample of BIOS in the parentboard

If the hardware is all working, BIOS will then look for a small sector at the very beginning of the primary hard disk called the Master Boot Record (MBR).

- o The MBR contains a list, or map, of all of the partitions on the computer's

hard disk (or disks).

After the MBR is found the Bootstrap Loader (an initial program stored in ROM) follows basic instructions for starting up the rest of the computer, including the operating system.

- o If multiple operating systems are installed, the user will be given a choice of which operating system to use.

The next stage is called Early Kernel Initialization. The Kernel is the core of the operating system, and it regulates all of the background functions of the computer.

- In the Early Kernel Initialization stage, a smaller core of the Kernel is activated.

- This core includes the device drivers needed to use the computer's RAM chips.

- o Without the extra memory provided by RAM, it is not possible to run the more complicated code for the remainder of the operating system.

The procedure of starting a computer by loading the kernel is known as booting the system.

Once the early Kernel Initialization is complete, the CPU switches to Protected Mode.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 21

CSC224: Operating System I

- The computer can now take advantage of the extended memory address system provided by RAM, and the operating system's Kernel is fully initialized.

- Only at this stage are the first User Mode processes initialized, and the user can begin interacting with the operating system, applications and hardware.

The diagram below shows the stages in starting an operating system.

Stages of starting an Operating System

Where are the Operating Systems Stored?

- Some systems — such as cellular phones, tablets, and game consoles — store the entire operating system in ROM (firmware).

- o Storing the operating system in ROM is suitable for small operating systems, simple supporting hardware, and rugged operation.

- o A problem with this approach is that changing the bootstrap code requires changing the ROM hardware chips.

- o A problem with firmware in general is that executing code there is slower than executing code in RAM.

- Some systems store the operating system in firmware and copy it to RAM for fast execution.

- For large operating systems (including most general-purpose operating systems like Windows, Mac OS X, and UNIX) or for systems that change frequently, the bootstrap loader is stored in firmware, and the operating system is on disk.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 22

CSC224: Operating System I

1.5 Classification of Operating System

Operating System can be classified according to:

- o how user programs are processed
- o the number of users and application they support
- o

1.5.1 Classification according to how user programs are processed

□ Batch Operating Systems: The common input devices were card readers and tape drives. The common output devices were line printers, tape drives, and card punches.

o The users of such systems did not interact directly with the computer systems. Rather, the user prepared a job—which consisted of the program, the data, and some control information about the nature of the job (control cards)—and submitted it to the computer operator.

o The job would usually be in the form of punch cards. At some later time (perhaps minutes, hours, or days), the output appeared. The output consisted of the result of the program, as well as a dump of memory and registers in case of program error.

To improve utilization, the concept of a batch OS was developed. . The central idea behind the simple batch-processing scheme is the use of a piece of software known as the monitor.

To process user programs,

- the monitor first has to be loaded into memory.
- Then it reads in programs one at a time from the input devices.
- As each program is read in, it will be placed in the user program area of main memory, and control is passed to this program.
- When the execution of the program is completed, it returns control to the monitor, which moves on to process the next program.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 23

CSC224: Operating System I

With this type of OS, the user no longer has direct access to the processor. Instead, the user submits the job on cards or tape to a computer operator, who batches the jobs together sequentially and places the entire batch on an input device, for use by the monitor.

o Each program is constructed to branch back to the monitor when it completes processing, at which point the monitor automatically begins loading the next program.

- The problems with Batch Systems include
- o lack of interaction between the user and job,

o and CPU is often idle because the speed of the mechanical I/O devices is slower than CPU.

□ Multi-programmed Operating Systems: To overcome the problem of underutilization of CPU and main memory, multiprogramming was introduced.

□ The multiprogramming is interleaved execution of multiple jobs by the same computer.

o when one program is waiting for I/O transfer; there is another program ready to utilize the CPU.

□ One problem with multiprogramming is that jobs that come late in the batch job list won't get chance to run until the jobs before them have completed,

o thus their users have to wait a long time to obtain the results.

□ Some programs may even need interaction with users, which requires the processor to switch to these programs frequently.

□ Different forms of multiprogramming operating system are multitasking, multi-process and multiuser operating systems

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 24

CSC224: Operating System I

OS features needed for multiprogramming:

- Memory management – the system must allocate the memory to several jobs
- Job Scheduling – Choose jobs in the job pool to bring them into memory
- CPU scheduling – the system must choose among several jobs ready to run
- Allocation of devices
- Protection (memory and OS protection)

Below is the memory layout for a multiprogramming system

0 Operating System

Job 1

Job 2

Job 3

Job 4

max

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 25

CSC224: Operating System I

□ Time-sharing Operating systems: A time-sharing system is multi-user,

multi-process, and interactive system.

o This means that it allows multiple users to use the computer simultaneously.

o In a time-sharing system, multiple users simultaneously access the system through terminals, with the OS interleaving the execution of each user program in a short burst or quantum of computation.

A user can run one or more processes at the same time and interact with his/her processes.

A time-shared system uses

□ multiprogramming (several programs run concurrently, interleaving their executions over time, which are perceived as simultaneous) and

□ CPU scheduling to provide each user with a small portion of a time-shared computer.

Each user has at least one separate program in memory. To obtain a reasonable response time, jobs may have to be swapped in and out of main memory.

o The Processor's time which is shared among multiple users simultaneously is termed as time-sharing.

o UNIX, Linux, Widows NT server, and Windows 2000 server are timesharing systems.

OS features needed for Time-Sharing include:

- Job Scheduling • Job synchronization and communication
- Memory management • Deadlock avoidance
- Process management • Allocation of devices
- CPU scheduling • Protection

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 26

CSC224: Operating System I

□ Distributed Systems: Distributed system is collection of loosely coupled processors interconnected by a communications network.

□ Distributed systems depend on networking for their functionality.

□ The nodes in a distributed system may vary in size and function. They may include small microprocessors, personal computers, and large general-purpose computer systems.

o These processors are referred to by a number of names, such as processors, sites, machines, and hosts, depending on the context in which they are mentioned.

o We mainly use site to indicate the location of a machine and node to refer to a specific system at a site.

Generally, one node at one site, the server, has a resource that another node at

another site, the client (or user), would like to use.

- There are four major reasons for building distributed systems:
 - o resource sharing, o reliability, and
 - o computation o communication
- speedup,

Distributed Computing Systems commonly use two types of Operating Systems –

- o distributed operating system and
- o Network operating. system

Distributed Operating Systems: A distributed operating system is for a multi-user operating system. Users can access other resources and computer on the network in a unified way.

- Every resource is visible on every site. A site is a specific location in a network containing two or more computer systems.
- A distributed operating system provides a less autonomous environment.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 27

CSC224: Operating System I

- o The different computers communicate closely enough to provide the illusion that only a single operating system controls the network.

- Distributed operating system allows programs to run on several processors at the same time.

- o Processors in a distributed system may vary in size and function. These processors are referred as sites, nodes, computers, and so on.

- When a program is executed on a distributed system, user is not aware of where the program is executed or the location of the resources accessed.

Network Operating System Network operating system is for a single-user operating system.

- Users are aware of other resources and computer on the network, and can access them by logging into the appropriate remote host.

- o A host is a specific computer system in the network that provides services.

- o A node is the name assigned to the host to identify it to other computers.

A network operating system is an operating system that provides features such as file sharing across the network, along with a communication scheme that allows different processes on different computers to exchange messages.

- A computer running a network operating system acts autonomously

from all other computers on the network,
o the users are aware of existence of multiple autonomous computers and can log in to remote machines and copy files from one machine to another machine.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 28

CSC224: Operating System I

A Network Operating System runs on a server and provides the server the capability to manage data, users, groups, security, applications, and other networking functions.

The primary purpose of the network operating system is to allow shared file and printer access among multiple computers in a network, typically a local area network (LAN), a private network or to other networks.

Some of typical characteristics of network operating systems which make it different from distributed operating system are:

- Each computer has its own private operating system instead of running part of a global system wide operating system.
- Each user normally works on his/her own system; using a different system requires some kind of remote login, instead of having the operating system dynamically allocate processes to CPUs.
- Users are typically aware of where each of their files are kept and must move file from one system to another with explicit file transfer commands instead of having file placement managed by the operating system.

Examples of network operating systems include Microsoft Windows Server 2003, Microsoft Windows Server 2008, UNIX, Linux, Mac OS X, Novell NetWare, and BSD (Berkeley System Distribution). Also, Android and iOS, are network operating systems.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 29

CSC224: Operating System I

- Embedded Operating System: Embedded operating systems are designed to be used in embedded computer systems.
- Embedded systems run on the computers that control devices that are not generally thought of as computers and which do not accept user-installed software.
- They are able to operate with a limited number of resources.
- Typical examples are microwave ovens, TV sets, cars, DVD recorders, cell phones, MP3 players.

The main property which distinguishes embedded systems from handhelds is the certainty that no un-trusted software will ever run on it.

o You cannot download new applications to your microwave oven—all the software is in ROM.

o This means that there is no need for protection between applications, leading to some simplification.

Windows CE and Minix 3 are some examples of embedded operating systems.

□ Real-Time Operating Systems

A real-time operating system is a multitasking operating system that aims at executing real-time applications.

□ Real-time systems are usually dedicated, embedded systems.

□ It is a special purpose operating system used when there are rigid time requirements on the operation of a processor or the flow of data,
o thus it is often used as a control device in dedicated applications such as Rocket launching, flight control, robotics, real time simulation, telephone switching equipments, medical imaging systems etc.

o Real time system requires that the processing must be done within the defined time constraints or the system will fail.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 30

CSC224: Operating System I

Real time systems are classified into two categories:

□ Soft Real time System: If certain deadlines are missed then system continues its working with no failure but its performance degrades.

□ Hard Real time System: If any deadline is missed then system will fail to work or does not work properly. This system guarantees that critical task is completed on time.

An operating system does the following activities related to real-time system activity:

- i. The Operating Systems typically read from and react to sensor data.
- ii. The Operating system must guarantee response to events within fixed periods of time to ensure correct performance.

1.5.2 Classification according to the number of users and application they support

□ Single-user, single-tasking - As the name implies, this operating system is designed to manage the computer so that one user can effectively do one thing at a time.

o The Palm OS for Palm handheld computers (e.g. cell phones) is a good example of a modern single-user, single-task operating system.

□ Single-user, multi-tasking - This is the type of operating system most people use on their desktop and laptop computers today.

o Microsoft's Windows and Apple's Mac OS platforms are both examples of operating systems that will let a single user have several

programs in operation at the same time.

o For example, it's entirely possible for a Windows user to be writing a note in a word processor while downloading a file from the Internet while printing the text of an e-mail message.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 31

CSC224: Operating System I

□ Multi-user - A multi-user operating system allows many different users to take advantage of the computer's resources simultaneously.

□ The operating system must make sure that the requirements of the various users are balanced, and that each of the programs they are using has sufficient and separate resources so that a problem with one user doesn't affect the entire community of users.

o Time-sharing system can be classified as multi-user systems as they enable a multiple user access to a computer through the sharing of time.

o UNIX, VMS (Virtual Memory System) and mainframe operating systems, such as MVS (Multiple Virtual Storage), are examples of multi-user operating systems.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 32

CSC224: Operating System I

2. Organization & Design of Operating System

2.1. Operating system functional components

An operating system has many components that manage all the resources in a computer system, ensuring proper execution of programs. We briefly describe these components in this section.

(i) Process manager

A process can be thought of as a program in execution. It needs certain resources, including CPU time, memory, files and I/O devices to accomplish its tasks. The operating system is responsible for:

Creating and terminating both user and system processes

Suspending and resuming processes

Providing mechanisms for process synchronization

Providing mechanisms for process communication

Providing mechanisms for deadlock handling

(ii) Main memory manager

Main memory is a large array of words or bytes (called memory locations), ranging in size from hundreds of thousands to billions.

- o Every word or byte has its own address. Main memory is a repository of quickly accessible data shared by the CPU and I/O devices.

- o It contains the code, data, stack, and other parts of a process. The central processor reads instructions of a process from main memory during the machine cycle— fetch-decode-execute.

The OS is responsible for the following activities in connection with memory management:

Keeping track of free memory space

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 33

CSC224: Operating System I

Keeping track of which parts of memory are currently being used and by whom

Deciding which processes are to be loaded into memory when memory space becomes available

Deciding how much memory is to be allocated to a process

Allocating and deallocating memory space as needed

Ensuring that a process is not overwritten by another

(iii) Secondary storage manager

The main purpose of a computer system is to execute programs. The programs, along with the data they access, must be in the main memory or primary storage during their execution.

- o Since main memory is too small to accommodate all data and programs, and

- o because the data it holds are lost when the power is lost, the computer system must provide secondary storage to backup main memory.

Most programs are stored on a disk until loaded into the memory and then use disk as both the source and destination of their processing. Like all other resources in a computer system, proper management of disk storage is important.

The operating system is responsible for the following activities in connection with disk management:

Free-space management

Storage allocation and deallocation

Disk scheduling

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 34

CSC224: Operating System I

(iv) I/O system manager

The I/O subsystem consists of:

A memory management component that includes buffering, caching and spooling

A general device-driver interface

Drivers for specific hardware devices

(v) File manager

Computers can store information on several types of physical media, e.g. magnetic tape, magnetic disk and optical disk. The OS maps files onto physical media and accesses these media via the storage devices.

The OS is responsible for the following activities with respect to file management:

Creating and deleting files

Creating and deleting directories

Supporting primitives (operations) for manipulating files and directories

Mapping files onto the secondary storage

Backing up files on stable (non-volatile) storage media

(vi). Protection system manager

If a computer system has multiple users and allows concurrent execution of multiple processes then the various processes must be protected from each other's activities. Protection is any mechanism for controlling the access of programs, processes or users to the resources defined by a computer system.

The protection mechanism must:

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 35

CSC224: Operating System I

Distinguish between authorized and unauthorized usage.

Specify the controls to be imposed.

Provide a means of enforcement.

(vii). Network manager

By creating a network, computers are able to share resources such as CPU, memory etc. There are two ways to configure the operating system to make it network enabled - these are network operating system and distributed operating system.

The operating system is responsible for:

Providing communication system between distributed processors.

Getting information about files/processes/etc. on a remote machine.

Below is a generalized diagram of the operating system organization

Operating System

Processes, Threads, File

Resources Manager Manager

Device

Memory Manager

Manager

Processor(s) Main Devices

Memory

Hardware

A generalized Operating System Organization

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 36

CSC224: Operating System I

2.2 Operating System Design Consideration

As it is the case of any non-trivial software, operating systems tend to be a complex combination of multiple components and many bindings between them. It is almost unthinkable to create such software just as a heap of unstructured code, without any explicit design. Therefore each operating system follows some kind of explicit design that can be captured more formally following design guidelines, principles and rules. In order to design a successful operating system, the designers have to have a clear idea of what they want. Lack of a goal makes it very hard to make subsequent decisions.

2.2.1 Operating System Design Goals

What do operating system designers want? It obviously varies from system to system, being different for embedded systems than for server systems. However, for general-purpose operating systems four main items come to mind:

Define abstractions.

Provide primitive operations.

Ensure isolation.

Manage the hardware.

Each of these items will be discussed below.

□ Define Abstraction: The most important, but probably hardest task of an operating system is to define the right abstractions i.e. hiding unnecessary details to reduce complexity.

o Some of them, such as processes, address spaces, and files, have been around so long that they may seem obvious.

o Others such as threads, are newer, and are less mature. For example, if a multithreaded process that has one thread blocked waiting for keyboard

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 37

CSC224: Operating System I

input forks, is there a thread in the new process also waiting for keyboard input?

- o Other abstractions relate to synchronization, signals, the memory model, modelling of I/O, and many other areas.

- Provision of Primitive Operations: Each of the abstractions can be instantiated in the form of concrete data structures. Users can create processes, files, semaphores, and more.

- o The primitive operations manipulate these data structures. For example, users can read and write files.

- o The primitive operations are implemented in the form of system calls.

- o From the user's point of view, the heart of the operating system is formed by the abstractions and the operations on them available via the system calls.

- Ensure Isolation: Since multiple users can be logged into a computer at the same time, the operating system needs to provide mechanisms to keep them separated.

- o One user may not interfere with another.

- o The process concept is widely used to group resources together for protection purposes.

- o Files and other data structures generally are protected as well.

- o Making sure each user can perform only authorized operations on authorized data is a key goal of system design.

- o However, users also want to share data and resources, so the isolation has to be selective and under user control.

- o This makes it much harder.

- o The e-mail program should not be able to clobber (damage) the Web browser.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 38

CSC224: Operating System I

- o Even when there is only a single user, different processes need to be isolated.

Closely related to this point is the need to isolate failures.

- o If some part of the system goes down, most commonly a user process, it should not be able to take the rest of the system down with it.

- o The system design should make sure that the various parts are well isolated from one another.

- o Ideally, parts of the operating system should also be isolated from one another to allow independent failures.

□ Hardware Management: Finally, the operating system has to manage the hardware..

o In particular, it has to take care of all the low-level chips, such as interrupt controllers and bus controllers.

o It also has to provide a framework for allowing device drivers to manage the larger I/O devices, such as disks, printers, and the display.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 39

CSC224: Operating System I

2.2.2 Operating System Design Principles

(i) Encapsulation Design Principle

The encapsulation principle guides in more detail how the individual components that are assembled into the entire operating system should be structured. The principle states:

- The interaction between components should always be implemented using a well-designed set of interfaces.
- The method of communication should be consistent for any given level of abstraction over the entire operating system.
- No interface and no method of communication should cross multiple levels of abstraction.

The purpose of this design principle is to force the developers to use generic design patterns instead of creating ad hoc solutions.

(ii) Portability Design Principle

The portability design principle guides the implementation decisions of OS, especially with respect to optimizations. The principle states:

- The design and implementation should always maintain a high level of platform neutrality and portability.
- The platform-specific code in the kernel, libraries and tasks should always be clearly separated from the platform-neutral code (either by a complete component decomposition or at least by internal compile-time APIs in the form of a hardware abstraction layer).

This design principle is a guard against optimizing the OS for any given target platform at the cost of making it highly suboptimal on some other platform.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 40

CSC224: Operating System I

o The practical benefit (except for generally more universal design) is to be ready to survive the downfall of the previously dominant hardware platform and to be able to be ported quickly on a new platform.

o Since an operating system represents a major software development effort, it is

desirable to make it easy to port it to different hardware platform.

The need for portability arises when hardware evolves e.g. Windows was first developed for the Intel 80386 platform, which has since evolved into 32 bit Pentium and now 64-bit platforms. Since no one makes 80386 based systems anymore,

□ Microsoft would be out of business if they did not port Windows to these new versions of the Intel CPU architecture!

(iii) Split of Mechanism and Policy Design Principle

The principle states:

- Whenever it is reasonable, each feature of the operating system should be decomposed into two sets of orthogonal aspects – one representing the basic mechanism of the feature and one representing the policy on which the mechanism is applied.
- If the mechanism requires it, it can be implemented in a component with a higher criticality level than the component implementing the policy.

A mechanism is a facility that physically enables a feature to be implemented, but does not contain any logic deciding on the conditions when the feature should be applied.

On the other hand, a policy is the logic deciding on the conditions when a feature should be applied, but without the facility that would physically implement the feature. In layman's terms, the mechanism is the "how" and the policy is the "when".

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 41

CSC224: Operating System I

The most important consequence is that the kernel component (as the part of the operating system running on the highest criticality level) should only implement mechanisms and not policies. The policies should be implemented in user space, whenever it is possible.

For example,

- o The timer construct is a mechanism for ensuring CPU protection, but deciding how long the timer is to be set for a particular user is a policy decision.
- o The protection of file through password is a mechanism, while the action to take when password fails is a policy

□ Why is the separation of mechanism and policy desirable?

□ Mechanism and policy must be separate to ensure that systems are easy to modify.

- o No two system installations are the same, so each installation may want to tune the operating system to suit its needs.

□ With mechanism and policy separate, the policy may be changed at will while the mechanism stays unchanged. This arrangement provides a more flexible system.

2.2.3 Why operating System is Hard to Design

(i) Operating systems have become extremely large programs. No one person can sit down at a PC and dash off a serious operating system in a few months. All current versions of UNIX exceed 3 million lines of code; Windows Vista has over 5 million lines of kernel code (and over 70 million lines of total code).

□ No one person can understand 3-5 million lines of code, let alone 70 million lines of code.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 42

CSC224: Operating System I

□ When you have a product that none of the designers can hope to fully understand, it should be no surprise that the results are often far from optimal.

(ii) Operating systems have to deal with concurrency. There are multiple users and multiple I/O devices all active at once.

□ Managing concurrency is inherently much harder than managing a single sequential activity.

(iii) Many users do want to share some of their information and resources with selected other users.

□ The operating system has to make this possible, but in such a way that malicious users cannot interfere.

(iv). Operating system designers really do not have a good idea of how their systems will be used, so they need to provide for considerable generality. Neither UNIX nor Windows was designed with e-mail or Web browsers in mind, yet many computers running these systems do little else.

(v) Modern operating systems are generally designed to be portable, meaning they have to run on multiple hardware platforms. They also have to support thousands of I/O devices, all of which are independently designed with no regard to one another

(vi). The frequent need to be backward compatible with some previous operating system. That system may have restrictions on word lengths, file names, or other aspects that the designers now regard as obsolete, but are stuck with.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 43

CSC224: Operating System I

2.3. Operating System Design Approaches

In this section, we discuss how the operating system components are interconnected and melded into a kernel. Kernel is that part of operating system which directly makes interface with hardware system. Some of its main functions are:

□ To provide a mechanism for creation and deletion of processes

□ To provide processor scheduling, memory management and I/O management

□ To provide mechanism for synchronisation of processes so that processes

synchronize their actions.

□ To provide mechanism for inter-process communication.

Various design approaches have been adopted, each approach producing a unique structure of the operating system. Some of these approaches include

- a. Monolithic e. Modules
- b. Layered f. Hybrid
- c. Microkernel g. Virtual machines
- d. Client – Server model

2.3.1 Simple (Monolithic) Structure Approach

In this approach, the entire operating system runs as a single program in kernel mode.

□ The kernel and device drivers are inside one address space running in kernel mode.

□ The operating system is written as a collection of procedures, linked together into a single large executable binary program.

□ When this technique is used, each procedure in the system is free to call any other one, if the latter provides some useful computation that the former needs.

□ In terms of information hiding, there is essentially none—every procedure is visible to every other procedure.

□ Examples include the MS-DOS and the traditional UNIX operating system

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 44

CSC224: Operating System I

Issues with this approach

□ Having thousands of procedures that can call each other without restriction often leads to an unwieldy and difficult to understand the system.

□ Also, it is hard to maintain, patch or extend. Bug fixing or adding new features requires the compilation of the whole source code which is a time and resource consuming process.

In MS-DOS, the interfaces and levels of functionality are not well separated. For instance, application programs are able to access the basic I/O routines to write directly to the display and disk drives.

o Such freedom leaves MS-DOS vulnerable to errant (or malicious) programs, causing entire system crashes when user programs fail.

o Of course, MS-DOS was also limited by the hardware of its era. Because the Intel 8088 for which it was written provides no dual mode and no hardware protection, the designers of MS-DOS had no choice but to leave the base hardware accessible.

Application Programming

Resident System Programming

MS-DOS drivers

ROM – BIOS Device Drivers

MS-DOS Structure

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 45

CSC224: Operating System I

The traditional UNIX operating system is layered to some extent.

It consists of two separable parts: the kernel and the system programs.

- o The kernel is further separated into a series of interfaces and device drivers, which have been added and expanded over the years as UNIX has evolved.

As shown in the diagram below, everything below the system-call interface and above the physical hardware is the kernel.

The kernel provides the file system, CPU scheduling, memory management, and other operating-system functions through system calls.

Traditional UNIX System Structure

2.3.2 Layered Structure Approach

The operating system is organized as a hierarchy of layers, each one constructed upon the one below it.

- The operating system is broken into a number of layers (levels).
- The bottom layer (layer 0) is the hardware; the highest (layer N) is the user interface.
- Higher level layer uses the functions (operations) and services of only lower-level layers.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 46

CSC224: Operating System I

The main advantage of the layered approach is the simplicity of construction and debugging. The layers are selected so that each uses functions (operations) and services of only lower-level layers.

- This approach simplifies debugging and system verification.
 - o The first layer can be debugged without any concern for the rest of the system, because, by definition, it uses only the basic hardware (which is assumed correct) to implement its functions.
 - o Once the first layer is debugged, its correct functioning can be assumed while the second layer is debugged, and so on.
- If an error is found during the debugging of a particular layer, the error must be on that layer, because the layers below it are already debugged.
 - o Thus, the design and implementation of the system are simplified.
- Each layer is implemented only with operations provided by lower-level layers.
 - o A layer does not need to know how these operations are implemented; it needs to know only what these operations do.

o Hence, each layer hides the existence of certain data structures, operations, and hardware from higher-level layers.

Below is a representation of a typical layered operating system

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 47

CSC224: Operating System I

2.3.3 Microkernel Structure Approach

This method structures the operating system by removing all nonessential components from the kernel and implementing them as system and user-level programs.

o The result is a smaller kernel.

□ The basic idea behind the microkernel design is to achieve high reliability by splitting the operating system up into small, well-defined modules, only one of which, the microkernel runs in kernel mode and the rest run as relatively powerless ordinary user processes.

□ In particular, by running each device driver and file system as a separate user process, a bug in one of these can crash that component, but cannot crash the entire system

o Thus a bug in the audio driver will cause the sound to be garbled or stop, but will not crash the computer.

o In contrast, in a monolithic system with all the drivers in the kernel, a buggy audio driver can easily reference an invalid memory address and bring the system to a grinding halt instantly.

□ There is little consensus regarding which services should remain in the kernel and which should be implemented in user space.

o Typically, however, microkernel provides minimal process and memory management, in addition to a communication facility.

The following diagram illustrates the architecture of a typical microkernel

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 48

CSC224: Operating System I

Benefits

□ One benefit of the microkernel approach is that it makes extending the operating system easier.

o All new services are added to user space and consequently do not require modification of the kernel.

o When the kernel does have to be modified, the changes tend to be fewer, because the microkernel is a smaller kernel.

o The resulting operating system is easier to port from one hardware design to another.

□ The microkernel also provides more security and reliability, since most

- services are running as user — rather than kernel — processes.
- o If a service fails, the rest of the operating system remains untouched.

Drawbacks

- Unfortunately, the performance of microkernel can suffer due to increased system-function overhead.
- o Consider the history of Windows NT. The first release had a layered microkernel organization.
- o This version's performance was low compared with that of Windows 95. Windows NT 4.0 partially corrected the performance problem by moving layers from user space to kernel space and integrating them more closely.
- o By the time Windows XP was designed, Windows architecture had become more monolithic than microkernel.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 49

CSC224: Operating System I

2.3.4 Client – Server Model

A slight variation of the microkernel idea is to distinguish two classes of processes,

- o the servers, each of which provides some service, and
- o the clients, which use these services.

This model is known as the client-server model. Often the lowest layer is a microkernel, but that is not required. The essence is the presence of client processes and server processes.

Communication between clients and servers is often by message passing. To obtain a service,

- o a client process constructs a message saying what it wants and sends it to the appropriate service.
- o The service then does the work and sends back the answer.
- o If the client and server happen to run on the same machine, certain optimizations are possible, but conceptually, we are still talking about message passing here.

An obvious generalization of this idea is to have the clients and servers run on different computers, connected by a local or wide-area network, as depicted in the diagram below.

Since clients communicate with servers by sending messages, the clients need not know whether the messages are handled locally on their own machines, or whether they are sent across a network to servers on a remote machine.

The client-server model over a network

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 50

CSC224: Operating System I

2.3.5 Modular Structure Approach

The current methodology for operating-system design involves using loadable kernel modules.

- Here, the kernel has a set of core components and links in additional services via modules, either at boot time or during run time.

- This type of design is common in modern implementations of UNIX, such as Solaris, Linux, and Mac OS X, as well as Windows.

- The idea of the design is for the kernel to provide core services while other services are implemented dynamically, as the kernel is running.

- Linking services dynamically is preferable to adding new features directly to the kernel,

- o which would require recompiling the kernel every time a change was made.

- Thus, for example, we might build CPU scheduling and memory management algorithms directly into the kernel and then add support for different file systems by way of loadable modules.

- The overall result resembles a layered system in that each kernel section has defined, protected interfaces;

- o but it is more flexible than a layered system, because any module can call any other module.

- The approach is also similar to the microkernel approach in that the primary module has only core functions and knowledge of how to load and communicate with other modules;

- o but it is more efficient, because modules do not need to invoke message passing in order to communicate.

The Solaris operating system structure, shown below is organized around a core kernel with seven types of loadable kernel modules:

Scheduling classes Loadable system calls

File systems Executable formats

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 51

CSC224: Operating System I

STREAMS modules Device and bus drivers

Miscellaneous

Solaris Loadable modules

2.3.6 Hybrid Structure Approach

In practice, very few operating systems adopt a single, strictly defined structure.

- o Instead, they combine different structures, resulting in hybrid systems that address performance, security, and usability issues.

- For example, both Linux and Solaris are monolithic, because having the operating

system in a single address space provides very efficient performance.

- o However, they are also modular, so that new functionality can be dynamically added to the kernel.

- Windows is largely monolithic as well (again primarily for performance reasons), but it retains some behaviour typical of microkernel systems, including providing support for separate subsystems (known as operating-system personalities) that run as user-mode processes.

- o Windows systems also provide support for dynamically loadable kernel modules.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 52

CSC224: Operating System I

- Other examples of hybrid systems include the Apple Mac OS X operating system and the two most prominent mobile operating systems — iOS and Android.

2.3.7 Virtual machine architecture Approach

- Most computers consist of multiple layers of hardware and software that operate together as a system.

- o Hardware resources typically include a central processor, display, storage, networking, and other peripheral devices.

- o Device drivers installed in the operating system control hardware resources, translating operating-system instructions into the specific device control language.

- o Device drivers assume exclusive device ownership. For example, it is assumed that a video driver owns the video adapter exclusively. Any software application that calls the video adapter must interact with the hardware through the video driver.

- Exclusive device ownership typically precludes running more than one operating system simultaneously on a computer.

- Virtual machine technology overcomes this limitation by redirecting interactions with device resources to lower system levels in such a way that higher-level application layers are unaffected.

- o In a Virtual Machine - each process "seems" to execute on its own processor with its own memory and devices i.e. the operating system creates the illusion of multiple processes, each executing on its own processor with its own (virtual) memory.

- The resources of the physical machine are shared. Virtual devices are sliced out of the physical ones. Virtual disks are subsets of physical ones.

- Useful for running different OS simultaneously on the same machine.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 53

CSC224: Operating System I

□ All virtual machines run within a single process. Each virtual processor runs on its own thread.

o All software code running within the virtual machine runs in a separate VMM (Virtual Machine Monitor) context.

□ The guest operating system and applications run on the virtual machine as if they were running on physical hardware, rather than emulated hardware.

o When a guest operating system is running, the special-purpose VMM kernel manages the CPU and hardware during virtual machine operations, creating an isolated environment in which the guest operating system and applications run close to the hardware at the highest possible performance.

Below is an example of MS-DOS on top of Windows XP.

Java Virtual Machine is another example of virtual machines as shown below

The Java Virtual Machine allows Java code to be portable between various hardware and OS platforms.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 54

CSC224: Operating System I

2.4. Mobile Operating Systems

The two most prominent mobile operating systems are iOS and Android. These operating systems are built on hybrid structures

2.4.1 iOS

The iOS is a mobile operating system designed by Apple to run its smart phone, the iPhone, as well as its tablet computer, the iPad. iOS is structured on the Mac OS X operating system, with added functionality pertinent to mobile devices, but does not directly run Mac OS X applications. The structure of iOS is shown below

Architecture of Apple's iOS

Cocoa Touch is an API for Objective-C that provides several frameworks for developing applications that run on iOS devices. The Cocoa Touch provides support for hardware features unique to mobile devices, such as touch screens.

The media services layer provides services for graphics, audio, and video.

The core services layer provides a variety of features, including support for cloud computing and databases.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 55

CSC224: Operating System I

2.4.2 Android

The Android operating system was designed by the Open Handset Alliance (led primarily by Google) and was developed for Android smart phones and tablet computers.

□ Whereas iOS is designed to run on Apple mobile devices and is close-sourced,

o Android runs on a variety of mobile platforms and is open-sourced, partly explaining its rapid rise in popularity.

□ Android is similar to iOS in that it is a layered stack of software that provides a rich set of frameworks for developing mobile applications.

At the bottom of this software stack is the Linux kernel, although it has been modified by Google and is currently outside the normal distribution of Linux releases. The structure of Android is shown in the diagram below.

Architecture of Google's Android.

□ Linux is used primarily for process, memory, and device-driver support for hardware and has been expanded to include power management.

□ The Android runtime environment includes a core set of libraries as well as the Dalvik virtual machine.

o Software designers for Android devices develop applications in the Java language.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 56

CSC224: Operating System I

o However, rather than using the standard Java API, Google has designed a separate Android API for Java development.

o The Java class files are first compiled to Java bytecode and then translated into an executable file that runs on the Dalvik virtual machine.

□ The Dalvik virtual machine was designed for Android and is optimized for mobile devices with limited memory and CPU processing capabilities.

□ The set of libraries available for Android applications includes

o frameworks for developing web browsers (webkit),

o database support (SQLite), and

o multimedia.

□ The libc library is similar to the standard C library but is much smaller and has been designed for the slower CPUs that characterize mobile devices.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 57

CSC224: Operating System I

3. Processes

3.1. What are Processes?

A Process is an instance of a program in execution. It is a powerful abstraction in Operating System design because:

□ It enables efficient management of running programs ensuring protection of one running program from another.

□ Multiple copies of the same program can be running simultaneously as individual process respectively.

□ It's a building block for building multi-programming and concurrent systems.

From an operating system design perspective, just as you have kernel space and user space, you also have kernel level process(es) running in the kernel space and user level process(es) running in the user space.

A process can use resources such as files in disk storage, input devices, output devices, CPU and so on.

These resources are made available to a process by the operating system

- o The operating system controls these resources
- o In fact in most cases, many processes share the above resources.
- o The operating system is in charge of ensuring optimal usage of these shared resources across processes

Food for thought: Think of the operating system like a government that controls resources of a nation.

A Simplistic View of A Process

When you double click (or tap for touch screens) on your favourite app (let's say firefox for illustration sake), the following happens:

- The operating system reads the program from disk memory (hard disk or memory card in phones),
- prepares it and loads it into main memory (RAM).
- The copy of the program in main memory is what is referred to as a process.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 58

CSC224: Operating System I

- Fun exercise: On windows or linux machine, do the following:
 - Open your favourite app
 - o On Windows, right click on the task bar and click on task manager and then click on Details tab to view running processes.
 - o On Linux (e.g. ubuntu), open system monitor from app menu, and then click on the processes tab.
 - Look through the list of processes and you will see the app you opened.
 - It is worth noting that each process is uniquely identified using a process id

3.2 Process Attributes

Process attributes are information that describes a particular process. Process attribute includes:

1. Identifier 5. Memory Pointers
2. State 6. Context Data
3. Priority 7. I/O Status Information
4. Program Counter 8. Accounting Information

Identifier: Each process is identified by its process identifier, (pid) which is a unique integer.

- o The pid provides a unique value for each process in the system, and it can be used as an index to access various attributes of a process within the kernel.

State: The state of a process is defined in part by the current activity of that process. The state may be new, ready, running, waiting, halted, and so on.

Program counter: The counter indicates the address of the next instruction to be executed for this process

Accounting information: This information includes the amount of CPU and real time used, time limits, account numbers, job or process numbers, and so on.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 59

CSC224: Operating System I

I/O status information: This information includes the list of I/O devices allocated to the process, a list of open files, and so on.

Memory-management information: This information may include such items as the value of the base and limit registers and the page tables, or the segment tables, depending on the memory system used by the operating system

Context Information: It includes the value of the CPU registers, the process and memory- management information

Priority: Represents an approximation of the relative importance of process. Process with a higher priority is placed on the high priority dispatch queue.

3.3 Process State

As a process executes, it changes state. The state of a process is defined in part by the current activity of that process.

Below is a state diagram showing the three states a process may be in:

1. Running (actually using the CPU at that instant).
2. Ready (runnable; temporarily stopped to let another process run).
3. Blocked (unable to run until some external event happens).

Three-state model of a Process

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 60

CSC224: Operating System I

Logically, the first two states are similar. In both cases the process is willing to run, only in the second one, there is temporarily no CPU available for it. The third state is fundamentally different from the first two in that the process cannot run, even if the CPU is idle and has nothing else to do.

Four transitions are possible among these three states, as shown.

□ Transition 1 occurs when the operating system discovers that a process cannot continue right now. In some systems the process can execute a system call, such as pause, to get into blocked state.

o In other systems, including UNIX, when a process reads from a pipe or special file (e.g., a terminal) and there is no input available, the process is automatically blocked.

□ Transitions 2 and 3 are caused by the process scheduler, a part of the operating

system, without the process even knowing about them. Transition 2 occurs when the scheduler decides that the running process has run long enough, and it is time to let another process have some CPU time.

□ Transition 3 occurs when all the other processes have had their fair share and it is time for the first process to get the CPU to run again.

□ Transition 4 occurs when the external event for which a process was waiting (such as the arrival of some input) happens.

o If no other process is running at that instant, transition 3 will be triggered and the process will start running.

o Otherwise it may have to wait in ready state for a little while until the CPU is available and its turn comes.

Using a five-state model, a process may be in one of the following states:

- New. The process is being created.
- Running. Instructions are being executed.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 61

CSC224: Operating System I

- Waiting. The process is waiting for some event to occur (such as an I/O completion or reception of a signal).
- Ready. The process is waiting to be assigned to a processor.
- Terminated. The process has finished execution.

These names are arbitrary, and they vary across operating systems. The states that they represent are found on all systems

o It is important to realize that only one process can be running on any processor at any instant.

o Many processes may be ready and waiting, however.

The state diagram corresponding to these states is presented in the diagram below

Diagram of Process Five-state Model

3.4 Process Control Block (PCB)

Each process is represented in the operating system by a process control block (PCB)—also called a task control block.

It contains many pieces of information associated with a specific process. These include process id, state, program counter, CPU registers, memory information and other process attributes discussed earlier.

o the PCB simply serves as the repository for any information that may vary from process to process.

A PCB is shown in the diagram below

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 62

CSC224: Operating System I

PCB is one of the most important data structure/component of an operating system.

The PCB is like a ledger where a “row in the list” contains information about corresponding process. This record is not accessible to the corresponding process or any other process

- o The Operating System stores a list of PCB (one for each process)
- o Thus, each process is unique to the operating system
- o The PCB enables to operating system to manage processes so that there are no chaos
- The difficulty in managing PCBs is not access to it, rather it is the protection of access to it
- o PCBs are read/modified by most modules in the operating system .
- o An error (bug) in one module/component in an operating system can damage the PCBs if accessed/modified incorrectly. This can hinder the operating system from properly managing affected process.
- o A design change in a structure or elements of a PCB can affect several modules/components in the operating system that read/modify PCBs.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 63

CSC224: Operating System I

3.5. Context Switching

Context Switching entails the operating system re-assigning the CPU from one process to another.

- o This is made possible because of the attribute “state” of a process.
- o It enhances efficient utilization of CPU time and other resources
- o It enables concurrency (executing multiple processes)

Switching the CPU to another process requires performing a state save of the current process and a state restore of a different process.

- o When a context switch occurs, the kernel saves the context of the old process in its PCB and loads the saved context of the new process scheduled to run.

Context-switch time is pure overhead, because the system does no useful work while switching.

- o Switching speed varies from machine to machine, depending on the memory speed, the number of registers that must be copied, and the existence of special instructions (such as a single instruction to load or store all registers).

The address space of the current process must be preserved as the space of the next task is prepared for use.

The diagram below shows the switching of CPU between processes

Diagram showing CPU switch from process to process

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 64

CSC224: Operating System I

3.6. Fundamentals of Process Scheduling

Process Scheduling is the steps involved in determining which process in the ready

queue gets assigned the CPU for execution.

In a general sense, apart from CPU resource scheduler, there are different schedulers for different system resources (e.g. file, memory, network and so on), not just CPU resource.

The part of the OS that handles process scheduling is called a Scheduler.

● There are different types of algorithms used to implement a scheduler. Examples include;

- First Come First Serve (FCFS)
- Shortest Job First
- Round Robin (RR)
- Priority scheduling
- etc

Details about these algorithms will be discussed in CSC312 (Operating System II)

● There is no optimal scheduling algorithm. The resource to allocate, task and other factors determine which scheduling algorithm is suitable for use.

A common representation of process scheduling is a queuing diagram, such as that is shown below

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 65

CSC224: Operating System I

Queuing-diagram representation of process scheduling

Each rectangular box represents a queue. Two types of queues are present: the ready queue and a set of device queues.

The circles represent the resources that serve the queues, and the arrows indicate the flow of processes in the system.

A new process is initially put in the ready queue. It waits there until it is selected for execution, or dispatched. Once the process is allocated the CPU and is executing, one of several events could occur:

- The process could issue an I/O request and then be placed in an I/O queue.
- The process could create a new child process and wait for the child's termination.
- The process could be removed forcibly from the CPU, as a result of an interrupt, and be put back in the ready queue.

In the first two cases, the process eventually switches from the waiting state to the ready state and is then put back in the ready queue.

o A process continues this cycle until it terminates, at which time it is removed from all queues and has its PCB and resources de-allocated.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 66

CSC224: Operating System I

3.7. Operations on Processes

The processes in most systems can execute concurrently, and they may be created and deleted dynamically. Thus, these systems must provide a mechanism for process creation and termination

3.7.1. Process Creation

During the course of execution, a process may create several new processes.

- o The creating process is called a parent process, and
- o the new processes are called the children of that process.
- o Each of these new processes may in turn create other processes, forming a tree of processes.

A process can be created by the OS as a result of different reasons. They include;

- o Spawned by existing process: a user process can initiate a request to create another process. The newly created
- o User Login: when a user logs in, some process(es) are created by the OS
- o OS Service: The OS can create a process to perform a function on behalf a user process (e.g. a process to provide printing service on the computer).

In general, when a process creates a child process, that child process will need certain resources (CPU time, memory, files, I/O devices) to accomplish its task.

- o A child process may be able to obtain its resources directly from the operating system, or it may be constrained to a subset of the resources of the parent process.

- o The parent may have to partition its resources among its children, or it may be able to share some resources (such as memory or files) among several of its children.

When a process creates a new process, two possibilities for execution exist:

- o The parent continues to execute concurrently with its children.
- o The parent waits until some or all of its children have terminated.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 67

CSC224: Operating System I

□ Steps involved in Process Creation

There are different steps involved in creating a process. These include;

- Assign a new process identifier to the new process.
- Allocate space for the process.
- Initialise the Process Control Block (PCB)
- Set the appropriate linkages

Process Creation in Windows OS

□ A process is created in windows using the function:

- CreateProcess(...)
- Creates a process from the process which it was called
- Returns boolean value; true if process (and thread) was created successfully, otherwise false is returned.

- It does not clone the parent process like unix/linux; rather, you specify the application program to be executed as a parameter

□ Include the header file windows.h in order to use the CreateProcess(...) method.

Process Creation in Unix/Linux OS

To create a process in Unix/Linux, use the function:

□ fork()

□ It creates a clone (child process) of the process (parent process) in which it was called

□ Returns 0 to the calling child process

□ Returns child process id to the parent process

□ If child process could not be created, then fork returns -1

Compiled by: Orhionkpaio, B. C Date: March 2021 Page 68

CSC224: Operating System I

3.7.2 Process Termination

A process terminates when it finishes executing its final statement and asks the operating system to delete it by using the exit() system call.

o At that point, the process may return a status value (typically an integer) to its parent process (via the wait() system call).

o All the resources of the process—including physical and virtual memory, open files, and I/O buffers - are de-allocated by the operating system.

A process can also terminate (end) for other number of reasons. They include;

□ Memory Unavailable: process requires more memory than the current available space in memory.

□ Bounds Violation: process tries to access memory location/region that it is not allowed to access (e.g. memory address space of another process, memory location of OS kernel and so on).

□ Protection Error: process tries to use a resource (e.g. file) that it is not allowed to use or use the resource inappropriately (e.g. writing to a read-only file).

□ Parent Process Termination: only affects processes created by another process. When the parent process terminates, the OS may terminate all child processes of the parent process.

□ User or OS Intervention: User exit the program or the OS forcefully terminates the process (for example, to resolve resource deadlock).

□ I/O Failure: Failure of an I/O device being used by process.

□ Invalid Instruction: process attempts to execute an invalid instruction (e.g. branching into data segment and then attempting to execute the data).

Compiled by: Orhionkpaio, B. C Date: March 2021 Page 69

CSC224: Operating System I

3.8 Fundamentals of Inter-Process Communication (IPC)

Inter-Process Communication (IPC) is the mechanism set up in place to enable two processes communicate with each other.

● Fundamental requirements for IPC that need to be satisfied;

- Synchronisation: to ensure information sent by sender process gets to the receiver process.
- Communication: exchange of information.

Processes executing concurrently in the operating system may be either independent processes or cooperating processes.

o A process is independent if it cannot affect or be affected by the other processes executing in the system. Any process that does not share data with any other process is independent.

o A process is cooperating if it can affect or be affected by the other processes executing in the system.

o Clearly, any process that shares data with other processes is a cooperating process..

There are several reasons for providing an environment that allows process cooperation:

□ Information sharing:.. Since several users may be interested in the same piece of information (for instance, a shared file), we must provide an environment to allow concurrent access to such information

□ Computation speedup:.. If we want a particular task to run faster, we must break it into subtasks, each of which will be executing in parallel with the others. Notice that such a speedup can be achieved only if the computer has multiple processing cores.

□ Modularity: We may want to construct the system in a modular fashion, dividing the system functions into separate processes or threads.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 70

CSC224: Operating System I

□ Convenience: Even an individual user may work on many tasks at the same time. For instance, a user may be editing, listening to music, and compiling in parallel.

Cooperating processes require an interprocess communication (IPC) mechanism that will allow them to exchange data and information.

There are two fundamental models of interprocess communication: shared memory and message passing.

□ In the shared-memory model, a region of memory that is shared by cooperating processes is established. Processes can then exchange information by reading and writing data to the shared region.

□ In the message-passing model, communication takes place by means of

messages exchanged between the cooperating processes.

- Message passing is useful for exchanging smaller amounts of data, because no conflicts need be avoided.
- Message passing is also easier to implement in a distributed system than shared memory.
- Shared memory can be faster than message passing, since message-passing systems are typically implemented using system calls

Communications models. (a) Message passing. (b) Shared memory

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 71

CSC224: Operating System I

3.8.1. Shared-Memory Systems

Inter-process communication using shared memory requires communicating processes to establish a region of shared memory.

Typically, a shared-memory region resides in the address space of the process creating the shared-memory segment.

Other processes that wish to communicate using this shared-memory segment must attach it to their address space.

- o Recall that, normally, the operating system tries to prevent one process from accessing another process's memory.
- o Shared memory requires that two or more processes agree to remove this restriction.
- o They can then exchange information by reading and writing data in the shared areas.
- o The processes are also responsible for ensuring that they are not writing to the same location simultaneously

3.8.2 Message-Passing Systems

Message passing provides a mechanism to allow processes to communicate and to synchronize their actions without sharing the same address space.

- o It is particularly useful in a distributed environment, where the communicating processes may reside on different computers connected by a network.
- o For example, an Internet chat program could be designed so that chat participants communicate with one another by exchanging messages.

A message-passing facility provides at least two operations:

send(destination, message) receive(Source, message)

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 72

CSC224: Operating System I

A process sends a message to another process (specified as destination parameter) by calling send(...).

While the receiving process receives the message from the sender (specified as

source) by calling receive(...).

If processes P and Q want to communicate, they must send messages to and receive messages from each other: a communication link must exist between them.

This link can be implemented in a variety of ways

- Direct or indirect communication
- Synchronous or asynchronous communication
- Automatic or explicit buffering

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 73

CSC224: Operating System I

3.8.3 Communication in Client/Server Systems

Client/Server paradigm is a computer networking concept where a client (computer) makes a request to access some form of resource on a server (computer) over a Computer Network.

The resource on the server could be files, CPU, memory and so on.

● Example of servers include;

- File server
- Web server
- Mail Server
- and so on

● One of the most popular client/server system is the web/internet.

A user access a website (e.g. "nairaland.com") from a client computer using web browser software (e.g. Opera) by making a request to an online server computer running a web server program.

□ The user's web browser is a process on the client computer. Also, the web server program in the server computer is a process.

□ Making a request means the client process sends a message (over the internet) to the server process.

□ The message is sent indirectly through mail box (port) on the server computer.

□ The server reads the message in the port, executes some instructions and sends a message (a web page) back to the client over the internet.

□ The message reaches the client's computer and it is stored in a mail box. The client process (web browser) reads the message and displays the web page on screen.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 74

CSC224: Operating System I

4. Threads

4.1 What are Threads?

A thread is a unit of execution within a process. Hence a single process can contain multiple threads, each carrying out different tasks.

You can consider a thread as a light weight process.

- Light weight because creation/management of threads does not require expensive operations when compared to process creation.
- All threads in a process share the same memory address space of the process to which it belongs. This is different from the process abstraction where each process has its own memory address space (even when the processes belong to the same application).
- In addition to sharing the memory address space to which a thread belongs, each thread also have their own unique allocation to store local thread variables and other information.

A traditional (or heavyweight) process has a single thread of control.

- o If a process has multiple threads of control, it can perform more than one task at a time.
- o Most software applications that run on modern computers are multithreaded.

A web browser might have one thread display images or text while another thread retrieves data from the network, for example.

- o A busy web server may have several (perhaps thousands of) clients concurrently accessing it. If the web server ran as a traditional single-threaded process, it would be able to service only one client at a time,
- o a client might have to wait a very long time for its request to be serviced.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 75

CSC224: Operating System I

A word processor may have a thread for displaying graphics, another thread for responding to keystrokes from the user, and a third thread for performing spelling and grammar checking in the background.

- An example of word processor with three threads (one each for spell checking, keyboard, disk backup) is shown in the diagram below

A word processor with three threads

4.2 Types of Threads

There are two major types of Threads

- User Level Thread
- Kernel Level Thread

Another type of thread is the combination of concepts from User Level Thread and Kernel Level Thread.

- Combined approach

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 76

CSC224: Operating System I

4.2.1 User Level Threads (ULTs)

Thread management is carried out by application running in the User Space of the

OS..

o ULTs provide a Library of functions to allow user processes to create and manage their own threads,

- Kernel is not aware of existence of thread.

- User threads are supported above the kernel and are managed without kernel support,

- Creation of new threads, switching, and synchronization are done without the intervention of the kernel

- Context switching for ULTs does not require the kernel space code execution.

- This saves time and allows context switching to be faster for ULTs than processes.

- Main advantage is the speed of execution of operations to manage thread. This is because it does not require any kernel space operation.

- Kernel space operations are expensive.

- The main drawback is that

- o when a thread is in a blocked state, then the process to which it belongs (and the other threads in the process) are placed in a blocked state.

- o Also, it does not thrive in multi-threaded processors since no more than one thread can be scheduled at the same time

Examples include Solaris 1 & 2, NT thread etc

The diagram below shows user level thread a

A User level Thread

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 77

CSC224: Operating System I

4.2.2 Kernel Level Threads (KLTs)

Threads within a process are managed by the OS kernel (in kernel space). The kernel provides system calls to create and manage threads.

- kernel threads are supported and managed directly by the operating system.

- The kernel has full knowledge of all threads

- Virtually all contemporary operating systems—including Windows, Linux, Mac OS X, and Solaris— support kernel threads.

- Context switching between threads will require kernel level operations.

- Advantage is that a thread in blocked state does not block the entire process to which it belongs.

- Main drawback is that since it requires kernel level operations, it means that its management is slower than that of ULTs.

- Kernel space operations are more expensive to process than user space operations.

- Increased in complexity since the kernel must manage and schedule threads

Examples include BSD, Solaris 2, Windows 95 and NT etc

Kernel Level Threads

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 78

CSC224: Operating System I

4.2.3 Combined Approach

This approach combines the ULTs and KLTs approaches.

- o Thread creation is carried out in the user space,
- o while other thread management activities are carried out in the kernel space.

Combined Approach

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 79

CSC224: Operating System I

4.3. Multi-Threading Models

A relationship exists between user threads and kernel threads. In this section, we look at three common ways of establishing such a relationship: the many-to-one model, the one-to-one model, and the many-to many models

4.3.1 Many-to-One Model

The many-to-one model (see diagram below) maps many user-level threads to one kernel thread.

- Thread management is done by the thread library in user space, so it is efficient.
 - However, the entire process will block if a thread makes a blocking system call.
 - Also, because only one thread can access the kernel at a time, multiple threads are unable to run in parallel on multicore systems.
- Green threads—a thread library available for Solaris systems and adopted in early versions of Java—used the many-to-one model.
- However, very few systems continue to use the model because of its inability to take advantage of multiple processing cores.

Many-to-one Model

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 80

CSC224: Operating System I

4.3.2 One-to-One Model

The one-to-one model maps each user thread to a kernel thread.

It provides more concurrency than the many-to-one model

- o by allowing another thread to run when a thread makes a blocking system call.
- o It also allows multiple threads to run in parallel on multiprocessors.

The only drawback to this model is that

- o creating a user thread requires creating the corresponding kernel thread.
- o Because the overhead of creating kernel threads can burden the performance of an application, most implementations of this model restrict the number of

threads supported by the system.

Linux, along with the family of Windows operating systems, implement the one-to-one model.

One-to-One Model

4.3.3. Many-to-Many Model

The many-to-many model multiplexes many user-level threads to a smaller or equal number of kernel threads.

- The number of kernel threads may be specific to either a particular application or a particular machine (an application may be allocated more kernel threads on a multiprocessor than on a single processor).

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 81

CSC224: Operating System I

Many-to-Many Model

One variation on the many-to-many model still multiplexes many user level threads to a smaller or equal number of kernel threads but also allows a user-level thread to be bound to a kernel thread.

- This variation is sometimes referred to as the two-level model
- The Solaris operating system supported the two-level model in versions older than Solaris 9.
- However, beginning with Solaris 9, this system uses the one-to-one model.

Two-Level Model

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 82

CSC224: Operating System I

4.4 Benefit of Threads

Whether ULTs or KLTs, the following are some benefit of threads over processes:

- Takes less time to create a new thread than a new process.
- Takes less time to terminate a thread than a process.
- Context switching between two threads takes less time than between two processes.
- Threads also play a vital role in remote procedure call (RPC) systems.
 - o When a server receives a message, it services the message using a separate thread.
 - o This allows the server to service several concurrent requests.

4.5 Issues with Threads

- Even though threads share address space of the process to which they belong, each thread contains a unique storage allocation for stack memory (for its own function call operations and local variables). This can cause problems as stated below.
 - o This additional memory may not necessarily be protected from

other threads in the process.

- o Hence the programmer has to ensure that threads only read-from or write-to the memory location that it is allocated (unless when there is need to share data)

- o Recall that memory allocated to a process is protected from other process by default.

- The idea of multiple threads (of same process) sharing its process data/variables is an advantage and a disadvantage

- Advantage because you don't need communication between threads (which is an added computation overhead as seen in inter-process communication) in order to share data.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 83

CSC224: Operating System I

- Disadvantage because allowing multiple threads to access same data/variable at an instance in time can lead to corruption of data which leads to data inconsistency or could possibly crash your program.

- o This is called a Race Condition

To summarise, if not properly managed, possible problems that can arise from using threads include:

- Lack of protection of unique memory address space allocated to thread.

Threads belonging to same process can write/read from each other's memory address space.

- Lack of protection of process memory address space to which threads belongs. All threads belonging to a process have equal access to the process' memory address space and can corrupt stored data as a result of concurrent execution of thread.

4.5.1 Threads and Race Conditions

A Race Condition occurs when two or more threads of a process read-from and write-to a process memory address space (e.g. a variable declared inside the process).

For demonstration purpose, consider the example below.

- Let's say we want a program to multiply a variable (called wage with current value as 50) by 2 and then add 100 to the result and then store the final result in the same variable. We can write the program to contain 2 threads. One thread for the multiplication task and the other for the addition task. If the program runs correctly, the final answer should be 200. However, there are different orders that the threads can run concurrently.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 84

CSC224: Operating System I

- If the threads are executed concurrently (without being programatically

careful), a possible execution path that can lead to corrupting the data stored in process variable wage is given below:

- Thread 1 reads the current value of wage (i.e. 50)
- Thread 2 reads the current value of wage. (i.e. 50)
- Thread 1 multiplies the read value by 2 (i.e. $50 \times 2 = 100$)
- Thread 2 adds 100 to the read value (i.e. $50 + 100 = 150$)
- Thread 1 writes result back to wage. variable (i.e. set wage to 100)
- Thread 2 writes result back to wage. variable (i.e. set wage to 150)

□ The result from the example (in the previous slide) is incorrect.

o The reason is because the two threads are simultaneously reading from and write to the shared process variable wage.

o For the program to work correctly, each thread 2 has to wait for thread 1 to finish its read from and writing to variable wage.

o This leads to the concept of “critical code section”.

4.5.2 Critical Code Section

A critical code section in a Multi-Thread program is a section of the program code that requires threads to be executed sequentially rather than concurrently.

□ This occurs (as seen in the example) when a thread tries to use a process variable that another thread is reading from or writing-to.

The concept of critical code section can be generalised broadly to computer resources and not just variables.

□ If a thread contains a program section that uses a resource (e.g. network, file read/write, process variable and so on) required by other threads, then that program section of thread is a critical code section.

The OS concept that ensures sequential execution of critical code section among threads (of a process) is called Thread Synchronisation.

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 85

CSC224: Operating System I

4.5.3 Thread Synchronisation

In order to prevent race conditions in critical code sections, we need to employ synchronisation techniques.

□ Synchronization meaning that when threads get to a critical code section, then they are executed sequentially (one after the other) and not concurrently (not parallel)

● Synchronisation techniques include:

- Mutex
- Semaphore
- Condition Variables

● The code for thread synchronisation is written by the programmer inside the Multi-Threaded program. This is why it was earlier stated that a programmer have to be

careful when using threads. The program is tasked with the responsibility to:

- Identify the critical code sections of the programs.
- Implement a thread synchronisation strategy/technique to combat race conditions.

Details on synchronization are discussed in CSC312 (Operating System II)

4.6 Processes vs Threads

Process Thread

Typically independent Exist as sub set of process

Have separate address space Share their address space

Context switch involves kernel and thus Faster context switch which is performed by slows down the process a user level library

Interacts through IPC (Inter-Process No need for IPC since threads can share Communication) only common data

Protected No protection between threads. It is quite possible that one thread overwrites the stack of another thread

Compiled by: Orhionkpaiyo, B. C Date: March 2021 Page 86